

工厂自动测试串口通讯协议

版次(Rev): 2.1.51

拟制 (Author) : 任立嘉

审核 (Checked By) :

批准 (Approved By) :

版本 Version	日期 Date	页码 Page	更新说明 Update Details
A0	2013-7-22	全部	首次发行, 实现小包传输通信
A1	2013-7-24	No.23	函数添加返回值
A2	2013-7-26		添加烧录 BARCODE 接口
A3	2013-7-27		添加 KSV 读取协议和 MAC 地址通信协议
A4	2013-8-23		美洲频道修正
A5	2013-9-29		添加版本信息
A6	2013-12-2		添加 CI 卡测试
A7	2013-12-19		添加 U 盘检查, WIFI, 以太网检查
2.0.0	2014-03-07		添加文件传输协议
2.1.0	2014-07-01		添加控制按键板功能 lock/unlock 命令
2.1.1	2014-10-30		添加 Panel model index 设置指令
2.1.2	2014-12-9		添加 TCL Set/Get Device ID 指令
2.1.3	2015-6-13		添加万能 Command Set/Get 和 USB 端口好坏检测指令

2.1.4	2015-6-29		更新 Command 万能接口，添加一位返回值类型 BYTE
2.1.5	2015-8-14		添加 SD 卡槽测试命令，在 UBS 状态上做扩展 添加 USB2.0 和 USB3.0 区分的功能 添加用户自定义烧录 bin 的功能 添加通用指令列表
2.1.6	2015-9-16		添加获取固定版本号的命令 添加通用指令表
2.1.6A	2015-11-9		更新获取 USB 设备状态说明
2.1.7	2015-11-30		新增烧录 WIFI, Bluetooth, Ethernet, Cus1, Cus2, Cus3 的 MAC 地址
2.1.7A	2015-12-8		修正了烧录 MAC 值组件，数据长度不正确的说明
2.1.7B	2016-8-9		添加固定通用指令序号表从 0x80 开始
2.1.8	2016-8-16		添加用户自定义 Serial Number 烧写指令，新增可选项 Cus1, Cus2, Cus3, Cus4, Cus5
2.1.8A	2016-8-17		修正获取用户 Serial Number 返回值的没有加 Type 的错误
2.1.9	2016-9-21		USB Status 的获取增加 WIFI 和 BT 属性，另外将最大数量扩充到 20
2.1.10	2016-11-23		增加获取老化模式时间的功能
2.1.11	2017-3-3		添加设置，获取 Project ID 的功能
2.1.12	2017-6-6		添加烧录 bin 时传输烧录文件名的功能
2.1.13	2017-11-20		添加音频测试功能
2.1.14	2018-4-17		添加工厂复位状态获取功能
2.1.15	2018-5-29		添加音频自动化测试的获取指令
2.1.15A	2018-5-29		更改音频自动化测试描述
2.1.16	2018-6-4		调整音频自动化测试获取参数功能，修改为带参获取的形式
2.1.17	2019-5-10		增加 Customer Barcode 定义，扩展烧录 WideVine 的名称
2.1.18	2020-6-15		调整 Customer Barcode 定义，Cus1 调整为客户条码
2.1.19	2020-7-21		调整文件传输协议 CUS4 为烧录一个本地固定 Key 描述
2.1.20	2021-2-7		规范文件传输协议 CUS1~CUS5 用途
2.1.21	2021-3-15		增加传输 Key 文件种类的描述，增加通道设置部分新增 4 个 Type-C 通道
2.1.22	2021-4-14		增加涂鸦 IotKey,增加远场语音测试指令
2.1.23	2021-4-21		删减自定义 Barcode 烧录定义，Key 文件烧录推荐使用文件传输指令
2.1.24	2021-6-21		修改背光设置命令错误描述，添加烧录 CI Plus 2ND Key 指令
2.1.25	2022-2-15		修正远场语音指令 id，添加按键板历史获取命令 添加烧录 Fairplay Key 指令

2.1.26	2022-3-24		增加 HDCPTX key 类型
2.1.27	2022-7-04		增加获取 CPU 温度的指令，增加 Xiao Mi 蓝牙搜索的指令，增加通用串口指令的返回值的判断类型，判定是否在预设的范围内
2.1.28	2022-8-12		增加 SMART HOME key 类型
2.1.29	2022- 11- 23		增加 MIRACAST key 类型
2.1.30	2023- 01- 06		增加 PEK 、YouTube 烧录类型
2.1.31	2023-02-07		增加通用串口指令返回值判断类型，判断返回字符串是否包含预设字符串
2.1.32	2023-03-30		增加 MCU 版本获取
2.1.33	2023-06-01		增加 EDV KEY、KFP、OTA Client 烧录类型
2.1.34	2023-06-02		新增本地 CommonType1、CommonType2、CommonType3、CommonType4 烧录类型
2.1.35	2023-06-09		新增 Customer Barcode 指令 CUS6 烧录类型
2.1.36	2023-06-20		增加 LEK KEY 烧录类型，和 pek key 配合，做 DHAv1
2.1.37	2023-07-04		增加获取所有 USB 口 U 盘名称指令
2.1.38	2023-07-06		增加 NabtoKey 烧录类型，特定客户(NEC)用于秘钥烧录
2.1.39	2023-08-10		1、增加 Tivo key 烧录类型，客户用于系统联网校验的，为了防止未烧录 tivokey 的板卡联入 tivo 内网 2、添加 PFID 和 PFPK key 烧录类型，加解密 MGKID key 使用
2.1.40	2023-10-25		增加 CVTE AT 中发送字符串并执行的指令
2.1.41	2023-12-05		增加 Marlin key 烧录类型，客户用于 HD+的 app，该 key 只有出德国才需要烧录
2.1.42	2024-1-10		增加核心板 Barcode 烧录指令(适合于核心板+底板情况)
2.1.43	2024-3-15		增加获取网络速率指令
2.1.44	2024-8-07		增加获取 RID 功能
2.1.45	2024-08-19		新增 hdmi6 和 hdmi7 通道枚举
2.1.46	2024-09-04		新增 3.5 节 ACK 应答指令和错误码表，删除第 4 章中的冗余描述
2.1.47	2025-04-25		4.27 文件传输指令添加 RMCA Key 和 CombinedKey 类型
2.1.48	2025-05-12		新增 4.49 文件回传指令
2.1.49	2025-05-13		4.48 和 4.49 文件回传指令和获取 RID 指令命令字冲突。 将 RID 命令字改成 A8、A9，放到 4.49 节
2.1.50	2025-06-06		获取通道 RGB 数据
2.1.51	2025-07-21		新增 4.51 烧录 Key 压缩包指令

一，概述

为规范电视的串口通讯协议制定的统一的底层数据包的格式协议。

二，范围

所有自定义的使用串口通讯协议。

三，标准内容

3.1 串口设置要求

- D =8
- P =0
- S =1
- 波特率： 115200
- 数据连续性要求，同一个包内的数据间隔不超过 100ms

3.2 设计规格

目前提供小包通信格式。小包长度为[6 - 255]

字节序号	值	描述
0x00	0xFF	同步字符
0x01	0x33	开始字符
0x02	0xNN (>=0x6)	包长度，必须等于或大于 6
0x03 ... 0xNN-2	0x00-0xFF	数据
0xNN-1	0x00-0xFF	Checksum

校验计算方式：（数据都使用无符号数）
Checksum =0x100-(Data[0x02]+ ...+ Data[0xNN – 2])

3.3 串口协议类型描述

数据部分的前 1-2 个字节为 协议命令字 PROTOCOL_ID。
第一个字节 最高位 bit7 为 1 时表示,协议命令字由两个字节组成。
后面为数据部分。
例： RF 测试 PROTOCOL_ID 为 0x01
包数据为 0x02， 0x00,0x01,0x02,0x03

3.4 协议类型列表 (Protocol id list)

协议命令字 Protocol id	协议名称	描述
0x01	RF 自动测试	
0x02	自动白平衡	
0x03	工厂自动测试	
0x04	自动测试信号发生器	

3.5 ACK 应答指令

返回 ACK 应答(TV->PC): 下文省略

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0C	包长度
03	03	工厂自动测试协议
04	01	ACK CMD ID
05	00-02	ACK ERROR CODE
06	0x00-0xFF	Cmd ID for ACK, 返回应答的 CMD ID
07	00	Pocket Idx (31:24)
08	00	Pocket Idx (23:16)
09	00	Pocket Idx (15:8)
0A	00	Pocket Idx (7:0)
0B	100-(data[2] + ... + data[0B - 1])	校验码

ACK ERROR CODE(错误码表)

值 (Value)	描述
00	正确 (OK)
01	命令无法识别 (Unknown CommandID)
02	参数错误 (Parameter error)

Pocket Idx 为回复的数据包 index, 没有时为 0。

四，工厂串口通信标准协议

4.1 命令总表

命令字	命令名称	方向	长度	版本	描述
0x00	START_HDCP	PC->TV	0A	1.0	开始烧录 HDCP 命令
0x01	ACK	TV->PC	0C	1.0	命令接收返回应答
0x02	RET_START_HDCP	TV->PC	09	1.0	返回能否烧录 HDCP KEY
0x03	SEND_HDCP_DATA	PC->TV		1.0	传输 hdcv key 数据
0x04	SEND_HDCP_CRC	PC->TV	08	1.0	发送 hdcv CRC 到 TV，如果正常就烧录
0x05	ACK_HDCP_STATUS	TV->PC	07	1.0	返回是否烧录成功
0x06	START_CI_PLUS	PC->TV	0A	1.0	开始烧录 CI+ KEY 指令
0x07	RET_START_CI_PLUS	TV->PC	09	1.0	返回能否烧录
0x08	SEND_CI_PLUS_DATA	PC->TV		1.0	传输数据
0x09	SEND_CIPUS_CRC	PC->TV	08	1.0	发送 CRC 到 TV，如果正常就烧录
0x0A	ACK_CI_PLUS_STATUS	TV->PC	07	1.0	返回烧录是否成功的标志
0x0B	SET_MAC_ADDR	PC->TV		1.0	设置 MAC 地址
0x0C	GET_MAC_ADDR	PC->TV		1.0	获取 MAC 地址
0x0D	RET_MAC_ADDR	TV->PC		1.0	返回 MAC 地址
0x0E	GET_HDCP_ID	PC->TV	06	1.0	获取 hdcv ID
0x0F	RET_HDCP_ID	TV->PC	0A	1.0	返回 hdcv ID
0x10	GET_MAX_LEN	PC->TV		1.0	获取指令支持的最大长度
0x11	RET_MAX_LEN	TV-PC		1.0	返回指令支持的最大长度
0x12	GET_CHECKSUM	PC->TV	06	1.0	获取 TVchecksum
0x13	RET_CHECKSUM	TV->PC		1.0	返回 tv checksum
0x14	GET_SOURCE	PC->TV	06	1.0	获取 TV 通道 ID
0x15	RET_SOURCE	TV->PC		1.0	返回 TV 通道 ID
0x16	SET_SOURCE	PC->TV	07	1.0	设置通道
0x17	SET_VOLUME	PC->TV	07	1.0	设置音量
0x18	SET_CHANNEL	PC->TV		1.0	设置频道
0x19	CHANGE_PRO_NUM	PC->TV	07	1.0	切换频道

0x1A	CTRL_IO	PC->TV	08	1.0	控制 IO 口
0x1B	GET_CIPLUS_ID	PC-TV	06	1.0	获取 CI+ 唯一 ID
0x1C	RET_CIPLUS_ID	TV->PC	0A	1.0	返回 CI+唯一 ID
0x1D	GET_KEYPAD_STATUS	PC->TV	06	1.0	获取按键板当前状态
0x1E	RET_KEYPAD_STATUS	TV->PC	07	1.0	返回按键板当前状态
0x1F	SET_BARCODE	PC->TV		1.0	烧录 barcode 条形码指令
0x20	GET_BARCODE	PC->TV	06	1.0	获取 barcode 的指令
0x21	RET_BARCODE	TV->PC		1.0	返回 barcode 条形码
0x22	GET_HDCP_KSV	PC->TV		1.0	获取 HDCP KSV 的指令
0x23	RET_HDCP_KSV	TV->PC		1.0	返回 HDCP KSV 指令
0x24	SET_ATSC_AIR_CABLE	PC->TV		1.0	设置 ATSC 板卡切换 AIR 和 CABLE
0x25	RET_ATSC_AIR_CABLE	TV->PC		1.0	相应 ATSC 切换 AIR/CABLE 指令，并返回当前切换的模式
0x26	SET_ATSC_PRO_NUM	PC->TV		1.0	设置 ATSC 频道: XX-XX
0x27	GET_TOOL_VERSION	PC->TV		1.0	告知 TV 板卡回发版本信息
0x28	RET_TOOL_VERSION	TV->PC		1.0	发送串口模块的版本信息
0x29	CHECK_CI_FUNCTION	PC->TV		1.0	发送检查 CI 卡功能的指令
0x30	RET_CI_FUNCTION_STATUS	TV->PC		1.0	TV 返回 CI 功能是否正常状态(必须是有 CI 卡插入才正常)
0x31	GET_IP_INFO	PC->TV		1.0	检查 IP 指令
0x32	RET_IP_INFO	TV->PC		1.0	返回 IP
0x33	GET_WIFI_STATUS	PC->TV		1.0	检查 WIFI 状态
0x34	RET_WIFI_STATUS	TV->PC		1.0	返回 WIFI 状态 0: 正常; 1: 正在检查; 2: 失败
0x35	GET_USB_COUNT	PC->TV		1.0	检查 USB 使用个数, U 盘挂载检查
0x36	RET_USB_COUNT	TV->PC		1.0	返回 U 盘挂载个数
0x37	SEND_CVTE_FAC_IR	PC->TV		1.0	发送 CVTE 工厂遥控器码值, 返回 ACK 后执行动作.目前只接受工厂遥控器码值
0x38	CHECK_BLUETOOTH	PC->TV		1.0	检查蓝牙测试接口。测试结果由 ACK 携带返回

0x39	RET_BLUETOOTH_STATUS	TV->PC		1.0	返回 BLUETOOTH 的测试结果 0: 正常; 1: 正在检查; 2: 失败
0x40	START_SEND_FILE	PC->TV		1.0	开始烧录文件命令, 给出文件类型
0x41	RET_START_SEND_FILE	TV->PC	09	1.0	返回能否烧录
0x42	SEND_FILE_DATA	PC->TV		1.0	传输 FILE 数据
0x43	SEND_FILE_CRC	PC->TV	08	1.0	发送 FILE CRC 到 TV, 如果正常就烧录
0x44	ACK_FILE_STATUS	TV->PC	07	1.0	返回 TV 板卡 FILE 是否烧录成功
0x45	GET_FILE_ID	PC->TV		1.0	获取 FILE ID 的指令
0x46	RET_FILE_ID	TV->PC		1.0	返回 FILE ID 指令
0x47	CTRL_KEYPAD_FUNC	PC->TV		1.0	控制 KEYPAD 逻辑功能开关
0x48	SET_PANEL_MODEL_ID	PC->TV		1.0	设置 Panel model index
0x49	GET_PANEL_MODEL_ID	TV->PC		1.0	获取 Panel model index
0x50	SET_TCL_DEVICE_ID	PC->TV		1.0	设置 TCL Device ID
0x51	GET_TCL_DEVICE_ID	PC->TV		1.0	获取 TCL Device ID
0x52	RET_TCL_DEVICE_ID	TV->PC		1.0	返回 TCL Device ID
0x53	GET_USB_DEVICES_STAT	PC->TV		1.0	获取所有可插 USB 端口状态
0x54	RET_USB_DEVICES_STAT	TV->PC		1.0	返回所有 USB 端口状态
0x55	SET_COMM_CMD	PC->TV		1.0	设置通用指令
0x56	RET_COMM_CMD	TV->PC		1.0	返回通用指令所获取的数据值
0x57	GET_FIXED_CHECKSUM	PC->TV	06	1.0	获取固定版本号 TV checksum
0x58	RET_FIXED_CHECKSUM	TV->PC		1.0	返回固定版本号 TV checksum
0x59	SET_CFG_MAC_ADDR	PC->TV		1.0	设置可配置 MAC 地址
0x5A	GET_CFG_MAC_ADDR	PC->TV		1.0	获取可配置 MAC 地址
0x5B	RET_CFG_MAC_ADDR	TV->PC		1.0	返回可配置 MAC 地址
0x5C	SET_CUSTOMER_BARCODE	PC->TV		2.1.8	烧录用户自定义 barcode 条形码指令
0x5D	GET_CUSTOMER_BARCODE	PC->TV	07	2.1.8	获取用户自定义 barcode 的指令

0x5E	RET_CUSTOMER_BARCODE	TV->PC		2.1.8	返回用户自定义 barcode 条形码
0x5F	GET_AGING_STATUS	PC->TV		2.1.10	获取板卡老化时间
0x60	RET_AGING_STATUS	TV->PC		2.1.10	板卡老化模式的时间和参数
0x61	SET_PROJECTID	PC->TV		2.1.8	设置 ProjectID 的指令
0x62	GET_PROJECTID	PC->TV	07	2.1.8	获取 ProjectID 的指令
0x63	RET_PROJECTID	TV->PC		2.1.8	返回 ProjectID 的指令
0x64	SET_BIN_NAME	PC->TV		2.1.12	烧录 bin 文件的名称
0x65	GET_BIN_NAME	PC->TV	07	2.1.12	获取 bin 文件的名称
0x66	RET_BIN_NAME	TV->PC		2.1.12	返回 bin 文件的名称
0x67	SET_AUDIO_PRO	PC->TV		2.1.13	音频测试功能通用指令
0x68	SET_BACKLIGHT	PC->TV		2.1.13	设置系统背光
0x69	GET_BACKLIGHT	PC->TV	07	2.1.13	获取系统背光
0x6A	RET_BACKLIGHT	TV->PC		2.1.13	返回系统背光
0x6B	GET_RESET_STATUS	PC->TV	06	2.1.14	获取工厂复位完成状态
0x6C	RET_RESET_STATUS	TV->PC	07	2.1.14	返回工厂复位完成状态
0x6D	GET_AUDIO_PRO	PC->TV		2.1.15	获取音频自动化测试的返回值
0x6E	RET_AUDIO_PRO	TV->PC		2.1.15	返回音频自动化测试的值
0x6F	GET_FFM_STATUS	PC->TV		1.0	检查远场语音模块测试状态
0x70	RET_FFM_STATUS	TV->PC		1.0	返回远场语音模块测试状态 0: 正常; 1: 正在检查; 2: 失败
0x71	GET_CPU_TEMP	PC->TV		2.1.27	获取老化过程中最大的 CPU 温度
0x72	RET_CPU_TEMP	TV->PC		2.1.27	返回老化过程中最大的 CPU 温度
0x73	START_XIAOMI_BT_SCAN	PC->TV		2.1.27	小米要求: 通过设置 MAC 来扫描对应的 BT 设置
0x86	GET_MCU_VERSION	PC->TV		2.1.32	获取 MCU 版本号
0x87	RES_MCU_VERSION	TV->PC		2.1.32	返回 MCU 版本号
0x97	GET_KEYPAD_HISTORY	PC->TV		2.1.25	获取按键板按键历史记录
0x98	RET_KEYPAD_HISTORY	TV->PC		2.1.25	返回按键板按键历史记录
0x99	GET_ALL_USB_NAME	PC->TV		2.1.37	获取所有 usb 口 U 盘名称

0x9A	RET_ALL_USB_NAME	TV->PC		2.1.37	返回所有 usb 口 U 盘名称
0x9B	SEND_SHELL_COMMAND	PC->TV		2.1.40	增加 CVTE AT 中发送字符串并执行的指令
0x9C	SET_COREBOARD_BARCODE	PC->TV		2.1.42	烧录核心板 barcode 条形码指令(0x1F 和 0x20 是区分为底板)
0x9D	GET_COREBOARD_BARCODE	PC->TV		2.1.42	获取核心板 barcode 条形码指令(0x1F 和 0x20 是区分为底板)
0x9E	RET_COREBOARD_BARCODE	TV->PC		2.1.42	返回核心板 barcode 条形码
0xA0	GET_ETH_SPEED	PC->TV		2.1.43	获取网络速率
0xA1	RET_ETH_SPEED	TV->PC		2.1.43	返回网络速率
0xA2	SEND_FILE_PATH	PC->TV		2.1.48	发送文件路径指令
0xA3	RET_FILE_PACKAGE_COUNT	TV->PC		2.1.48	返回文件数据包数量
0xA4	READ_FILE_DATA	PC->TV		2.1.48	读取文件数据包
0xA5	RET_FILE_DATA	TV->PC		2.1.48	返回当前索引的数据包
0xA6	READ_FILE_CRC	PC->TV		2.1.48	读取原始文件 CRC
0xA7	RET_FILE_CRC	TV->PC		2.1.48	返回原始文件 CRC, PC 做校验
0xA8	GET_RID	PC->TV		2.1.49	获取 RID
0xA9	RET_RID	TV->PC		2.1.49	返回 RID
0xAA	GET_RGB_DATA	PC->TV		2.1.50	获取通道 RGB 值
0xAB	RET_RGB_DATA	TV->PC		2.1.50	返回通道 RGB 值
0xAC	START_SEND_COMBINE_KEY	PC->TV		2.1.51	开始传输 Key 压缩包指令
0xAD	RET_START_SEND_COMBINE_KEY	TV->PC		2.1.51	返回是否支持烧录 Key 压缩包指令
0xAE	SEND_COMBINE_KEY_DATA	PC->TV		2.1.51	传输 Key 压缩包指令
0xAF	ACK_COMBINE_KEY_STATUS	TV->PC		2.1.51	传输 Key 压缩包指令

4.2 获取 TV Checksum

注：后续均为 16 进制

获取命令(PC->TV):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度

03	03	工厂自动测试协议
04	12	获取 checksum id
05	E5	校验码

返回命令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	13	返回 checksum
05...(NN-2)	00-FF	CHECKSUM 数据
NN-1	100-(data[2] + ... + data[NN-2])	校验码

流程:

步骤	PC	TV
1	GET_CHECKSUM	
2		RET_CHECKSUM

4.3 获取 TV 通道 ID

获取 TV 通道指令(PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	14	获取通道 ID 指令
05	E3	校验码

返回 TV 通道指令(TV->PC)

字节序号	值	描述
00	FF	同步字

01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	15	返回通道 ID 的命令
05	00-FF	SOURCE ID
06	100-(data[2] + ... + data[06-1])	校验码

流程：

步骤	PC	TV
1	GET_SOURCE_ID	
2		RET_SOURCE_ID

通道 ID 枚举：

```
typedef enum {
    EN_AT_ATV = 0,
    EN_AT_DTV,
    EN_AT_DVBS,
    EN_AT_DVBC,
    EN_AT_DVBT,
    EN_AT_DVBT2,
    EN_AT_VGA,
    EN_AT_VGA2,
    EN_AT_HDMI1,
    EN_AT_HDMI2,
    EN_AT_HDMI3,
    EN_AT_HDMI4,
    EN_AT_HDMI5,
    EN_AT_SCART1,
    EN_AT_SCART2,
    EN_AT_AV1,
    EN_AT_AV2,
    EN_AT_AV3,
    EN_AT_AV4,
    EN_AT_YPBPR1,
```

```

EN_AT_YPBPR2,
EN_AT_YPBPR3,
EN_AT_YPBPR4,
EN_AT_USB1,
EN_AT_USB2,
EN_AT_USB3,
EN_AT_USB4,
EN_AT_SVIDEO1,
EN_AT_SVIDEO2,
EN_AT_DVD,
EN_AT_OTHER,
EN_AT_VGA3,
EN_AT_VGA4,
EN_AT_Type-C1,
EN_AT_Type-C2,
EN_AT_Type-C3,
EN_AT_Type-C4,
EN_AT_HDMI6,
EN_AT_HDMI7,
EN_AT_SOURCE_MAX,
}en_CVTE_AT_SOURCE;

```

4.4 设置 TV 通道

设置通道指令(PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	16	设置通道 ID 指令
05	00-FF	待设置的通道 ID
06	100-(data[2] + ... + data[06-1])	校验码

返回 ACK 应答(TV->PC)

流程:

步骤	PC	TV
1	SET_SOURCE_ID	
2		ACK

4.5 设置音量

设置音量命令(PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	17	设置音量指令
05	00-64	音量值
06	$100-(data[2] + \dots + data[06-1])$	校验码

返回 ACK 应答(TV->PC)

流程:

步骤	PC	TV
1	SET_VOLUME	
2		ACK

4.6 切换频道

切换频道指令(PC->TV):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	08	包长度
03	03	工厂自动测试协议
04	19	切换频道指令
05	00-FF	频道数[D15-D8]

06	00-FF	频道数[D7-D0]
07	100-(data[2] + ... + data[06-1])	校验码

返回 ACK 应答(TV->PC)
流程:

步骤	PC	TV
1	CHANGE_PRO_NUM	
2		ACK

4.7 控制 IO 口

设置命令(PC->TV):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	08	包长度
03	03	工厂自动测试协议
04	1A	设置的 IO 指令
05	00-FF	待设置的 IO ID
06	00-01	00 代表拉低电平, 01 代表拉高电平
07	100-(data[2] + ... + data[07-1])	校验码

返回 ACK 应答(TV->PC)
IO ID 枚举:

```
typedef enum {
```

```
    EN_AT_LVDS_PIN = 0,
```

```
    EN_AT_BLON_PIN,
```

```
    EN_AT_ADJ_PWM_PIN,
```

```
    EN_AT_DVD5V_PIN,
```

```
    EN_AT_DVDIR_PIN,
```

```
    EN_AT_DVD12V_PIN,
```

EN_AT_DVDDAT_PIN,

EN_AT_IO_MAX,

}en_CVTE_AT_IOPin;

IO 口电平控制参数:

typedef enum {

EN_AT_LOW = 0,

EN_AT_HIGH,

EN_AT_LEVEL_MAX,

//非法

}en_CVTE_AT_IO_LEVEL;

流程:

步骤	PC	TV
1	CTRL_IO	
2		ACK

注意: 有些板卡由硬件因素会使 IO 口电平与软件设置相反, 需要取反处理, 如 ADJ

4.8 烧录 HDCP KEY

开始烧录 HDCP 指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0A	包长度
03	03	工厂自动测试协议
04	00	开始烧录 HDCP KEY 的 CMD
05	00-FF	HDCP FILE ID [31 : 24]
06	00-FF	HDCP FILE ID [23 : 16]
07	00-FF	HDCP FILE ID [15 : 8]
08	00-FF	HDCP FILE ID [7 : 0]

09	100-(data[2] + ... + data[09-1])	校验码
----	----------------------------------	-----

返回能否烧录指令 (TV->PC)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	09	包长度
03	03	工厂自动测试协议
04	02	返回 HDCP KEY 开始烧录的应答 CMD
05	00-01	HDCP_STATUS (0:NORMAL,1:ALREADY EXIST)
06	00-FF	MAX PACKET LENGTH [15:8]
07	00-FF	MAX PACKET LENGTH [7:0]
08	100-(data[2] + ... + data[08-1])	校验码

传输数据指令(PC->TV):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	03	HDCP KEY 数据包 CMD
05	00-FF	POCKET INDEX [31 : 24]
06	00-FF	POCKET INDEX [23 : 16]
07	00-FF	POCKET INDEX [15 : 8]
08	00-FF	POCKET INDEX [7 : 0]
09	00-FF	Pocket Total count [31 : 24]
0A	00-FF	Pocket Total count [23 : 16]
0B	00-FF	Pocket Total count [15 : 8]
0C	00-FF	Pocket Total count [7 : 0]
0D	00-FF	DATA

...	...	...
NN-2	00-FF	DATA
NN-1	100-(data[2] + ... + data[NN-2])	校验码

对接收到的数据进行应答 ACK 指令 (TV->PC) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0C	包长度
03	03	工厂自动测试协议
04	01	ACK CMD ID
05	00-02	ACK ERROR CODE
06	03	Cmd ID for ACK, 返回应答的 CMD ID
07	00-FF	Pocket Idx (31:24)
08	00-FF	Pocket Idx (23:16)
09	00-FF	Pocket Idx (15:8)
0A	00-FF	Pocket Idx (7:0)
0B	100-(data[2] + ... + data[0B - 1])	校验码

ACK 返回的 Pocket idx 为接收到的包序号。

发送 HDCP KEY CRC 指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	08	包长度
03	03	工厂自动测试协议
04	04	发 hdcp key crc 的 CMD
05	00-FF	CRC [15 : 8]
06	00-FF	CRC [7 : 0]
07	100-(data[2] + ... + data[07 - 1])	校验码

返回烧录结果指令 (TV->PC) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	05	返回 hdcp key 烧录状态的 CMD
05	00-02	HDCP KEY STATUS(0: OK, 1: CRC ERROR, 2:write flash error)
06	100-(data[2] + ... + data[06 - 1])	校验码

流程:

步骤	PC	TV	工作内容
1	START_HDCP_KEY		传送 hdcp key 文件编码 ID, TV 收到后会判断是否烧录可以烧录 KEY, 如果可以, 申请空间, 然后由步骤 2 返回应答
2		RET_START_HDCP_KEY	返回: 能否烧录以及一个 pocket 最多可以传多少 byte 的 KEY 数据
3	SEND_KEY_DATA		传输数据
4		ACK	返回是否正确以及当前时第几个 pocket
...	...	...	...
N	SEND_KEY_DATA		
N+1		ACK	
N+2	SEND_KEY_CRC		当 n+1 的 ACK 返回的当前包等于总包数时, 表示发完数据, PC 发 CRC 码到 TV, TV 校验, 如果一致, 则写 flash
N+3		ACK_HDCP_STATUS	返回: 1, CRC 是否一直 2, 写入是否成功

4.9 获取 HDCP KEY FILE ID

获取 TV HDCP KEY FILE ID 的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	0E	获取 hdcp key CMD
05	100-(data[2] + ... + data[05 - 1])	校验码

返回 hdcp key id 指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0A	包长度
03	03	工厂自动测试协议
04	0F	返回 hdcp key file id
05	00-FF	HDCP FILE ID [31 : 24]
06	00-FF	HDCP FILE ID [23 : 16]
07	00-FF	HDCP FILE ID [15 : 8]
08	00-FF	HDCP FILE ID [7 : 0]
09	100-(data[2] + ... + data[09 - 1])	校验码

流程:

步骤	PC	TV
1	获取 hdci key file id	
2		返回 hdcp key file id

4.10 烧录 CI+KEY

开始烧录 CI+ KEY 指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0A	包长度

03	03	工厂自动测试协议
04	06	开始烧录 CI+ KEY 的 CMD
05	00-FF	CI+ ID [31 : 24]
06	00-FF	CI+ ID [23 : 16]
07	00-FF	CI+ ID [15 : 8]
08	00-FF	CI+ ID [7 : 0]
09	100-(data[2] + ... + data[09-1])	校验码

返回能否烧录指令 (TV->PC)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	09	包长度
03	03	工厂自动测试协议
04	07	返回 CI+ KEY 开始烧录的应答 CMD
05	00-01	CI+_STATUS (0:NORMAL,1:ALREADY EXIST)
06	00-FF	MAX PACKET LENGTH [15:8]
07	00-FF	MAX PACKET LENGTH [7:0]
08	100-(data[2] + ... + data[08-1])	校验码

传输数据指令(PC->TV):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	08	CI+ KEY 数据包 CMD
05	00-FF	POCKET INDEX [31 : 24]
06	00-FF	POCKET INDEX [23 : 16]
07	00-FF	POCKET INDEX [15 : 8]
08	00-FF	POCKET INDEX [7 : 0]
09	00-FF	Pocket Total count [31 : 24]

0A	00-FF	Pocket Total count [23 : 16]
0B	00-FF	Pocket Total count [15 : 8]
0C	00-FF	Pocket Total count [7 : 0]
0D	00-FF	DATA
...	...	...
NN-2	00-FF	DATA
NN-1	100-(data[2] + ... + data[NN-2])	校验码

对接收到的数据进行应答 ACK 指令 (TV->PC) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0C	包长度
03	03	工厂自动测试协议
04	01	ACK CMD ID
05	00-02	ACK ERROR CODE
06	08	Cmd ID for ACK, 返回应答的 CMD ID
07	00-FF	Pocket Idx (31:24)
08	00-FF	Pocket Idx (23:16)
09	00-FF	Pocket Idx (15:8)
0A	00-FF	Pocket Idx (7:0)
0B	100-(data[2] + ... + data[0B - 1])	校验码

发送 CI+ KEY CRC 指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	08	包长度
03	03	工厂自动测试协议
04	09	发 ci+ key crc 的 CMD
05	00-FF	CRC [15 : 8]
06	00-FF	CRC [7 : 0]
07	100-(data[2] + ... + data[07 - 1])	校验码

返回烧录结果指令（TV->PC）：

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	0A	返回 ci+ key 烧录状态的 CMD
05	00-02	CI+ KEY STATUS(0: OK, 1: CRC ERROR, 2:write flash error)
06	100-(data[2] + ... + data[06 - 1])	校验码

流程：

步骤	PC	TV	工作内容
1	START_CI_PLUS_KEY		传送 ci+ key 文件编码 ID, TV 收到后会判断是否烧录可以烧录 KEY, 如果可以, 申请空间, 然后由步骤 2 返回应答
2		RET_START_CIPPLUS_KEY	返回: 能否烧录以及一个 pocket 最多可以传多少 byte 的 KEY 数据
3	SEND_KEY_DATA		传输数据
4		ACK	返回是否正确以及当前时第几个 pocket
...	...	...	...
N	SEND_KEY_DATA		
N+1		ACK	
N+2	SEND_KEY_CRC		当 n+1 的 ACK 返回的当前包等于总包数时, 表示发完数据, PC 发 CRC 码到 TV, TV 校验, 如果一致, 则写 flash
N+3		ACK_CIPPLUS_STATU S	返回: 1, CRC 是否一直 2, 写入是否成功

4.11 获取 CI+KEY ID

获取 TV CI+ KEY ID 的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	1B	获取 CI+ key CMD
05	100-(data[2] + ... + data[05 - 1])	校验码

返回 ci+ key id 指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0A	包长度
03	03	工厂自动测试协议
04	1C	返回 CI+ key id
05	00-FF	CI+ KEY ID [31 : 24]
06	00-FF	CI+ KEY ID [23 : 16]
07	00-FF	CI+ KEY ID [15 : 8]
08	00-FF	CI+ KEY ID [7 : 0]
09	100-(data[2] + ... + data[09 - 1])	校验码

流程:

步骤	PC	TV
1	获取 CI+ key file id	
2		返回 CI+ key file id

4.12 烧录 MAC 地址

MAC 地址为: D₀ : D₁ : D₂ : D₃ : D₄ : D₅,

烧录的格式为: D₀D₁D₂D₃D₄D₅

烧录指令 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0C	包长度
03	03	工厂自动测试协议
04	0B	写入 MAC 的 CMD
05	00-FF	D ₀
06	00-FF	D ₁
07	00-FF	D ₂
08	00-FF	D ₃
09	00-FF	D ₄
0A	00-FF	D ₅
0B	100-(data[2] + ... + data[0B - 1])	校验码

流程:

步骤	PC	TV
1	烧录指令 MAC	
2		返回 ACK

4.13 获取 MAC 地址

获取 MAC 地址指令 (PC-TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	0C	获取 MAC 的指令
05	100-(data[2] + ... + data[05 - 2])	校验码

返回 MAC 指令 (TV->PC) :

字节序号	值	描述
00	FF	同步字
01	33	开始字

02	0C	包长度
03	03	工厂自动测试协议
04	0D	返回 MAC 的 CMD
05	00-FF	D ₀
06	00-FF	D ₁
07	00-FF	D ₂
08	00-FF	D ₃
09	00-FF	D ₄
0A	00-FF	D ₅
0B	100-(data[2] + ... + data[0B - 1])	校验码

流程:

步骤	PC	TV
1	获取 MAC 指令	
2		返回 MAC 指令

4.14 获取按键板状态

获取按键板状态的指令(PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	1D	获取 KEYPAD 状态的 CMD
05	100-(data[2] + ... + data[06 - 1])	校验码

返回按键板状态的指令(TV->PC)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	1E	返回 KEYPAD 状态的 CMD

05	00-FF	该字节的 D7-D0 分别代表 K7-K0 的状态, 1 代表按下, 0 代表未按下
06	100-(data[2] + ... + data[06 - 1])	校验码

流程:

步骤	PC	TV
1	发送获取按键板状态 KEYPAD_STATUS	
2		返回 KEYPAD_STATUS

4.15 烧录 Barcode 条形码

注意: BARCODE 的长度不固定

设置 Barcode 指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	1F	设置 bacode 的指令
05	XX	barcode-data
...	...	...
NN-1	XX	barcode-data
NN	100-(data[2] + ... + data[NN - 1])	校验码

返回 ACK 应答(TV->PC)

流程:

步骤	PC	TV
1	设置 barcode	
2		返回 ACK

4.16 获取 Barcode 条形码

获取 barcode 的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	20	获取板卡 BARCODE 的 CMD
05	$100-(data[2] + \dots + data[06 - 1])$	校验码

返回 BARCODE 的指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	21	设置 barcode 的指令
05	XX	barcode-data
...	...	...
NN-1	XX	barcode-data
NN	$100-(data[2] + \dots + data[NN - 1])$	校验码

获取流程

步骤	PC	TV
1	获取 barcode	
2		返回 barcode

4.17 获取 HDCP KSV 值

KSV 的标准为: 40 个 bits, “1”和“0” 各 20 个。

发送获取 KSV 的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字

01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	22	获取 KSV 的指令
05	100-(data[2] + ... + data[05 - 1])	校验码

返回 KSV 内容

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0B	包长度
03	03	工厂自动测试协议
04	23	设置 bacode 的指令
05	00-FF	KSV [39:32]
06	00-FF	KSV [31:24]
07	00-FF	KSV [23:16]
08	00-FF	KSV [15:8]
09	00-FF	KSV [7:0]
0A	100-(data[2] + ... + data[0A- 1])	校验码

流程:

步骤	PC	TV
1	获取 KSV	
2		返回 KSV

4.18 设置 ATSC 模式

设置 ATSC 的 AIR 或 CABLE 模式 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	24	设置 ATSC 模式

05	00 – 01	0: 为 AIR; 1: 为 CABLE
06	100-(data[2] + ... + data[06 – 1])	校验码

返回板卡当前切换的模式 (TV->PC)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	25	返回 ATSC 模式
05	00 – 01	0: 为 AIR; 1: 为 CABLE
06	100-(data[2] + ... + data[06 – 2])	校验码

步骤	PC	TV
1	SET_ATSC_AIR_CABLE	
2		RET_ATSC_AIR_CABLE

4.19 设置 ATSC 频点

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	08	包长度
03	03	工厂自动测试协议
04	26	设置 ATSC 频点的 ID
05	00-FF	主频道号
06	00-FF	次频道号
07	100-(data[2] + ... + data[07 – 2])	校验码

返回 ACK 应答(TV->PC)

流程:

步骤	PC	TV
1	SET_ATSC_PRO_NUM	

2		ACK
---	--	-----

4.20 获取串口模块版本信息

发送获取 TV 串口版本信息的 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	27	获取串口模块版本
05	$100-(data[2] + \dots + data[05 - 1])$	校验码

返回串口库版本(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	09	包长度
03	03	工厂自动测试协议
04	28	返回串口库版本
05	00-FF	版本 MAJOR
06	00-FF	版本 MINOR
07	00-FF	版本 PATCH
08	$100-(data[2] + \dots + data[08 - 1])$	校验码

流程:

步骤	PC	TV
1	获取版本信息	
2		返回版本信息

4.21 检查 TV CI 卡是否插入功能

发送获取 TVCI 状态 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	29	发送获取检查 CI 卡状态的命令
05	100-(data[2] + ... + data[05 - 2])	校验码

返回 CI 卡检查结果(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	30	CI 卡状态返回命令 ID
05	00-FF	0: 代表正常; 1: 代表未插入; 2: 代表 CI 卡与主板通信异常
06	100-(data[2] + ... + data[08 - 2])	校验码

流程:

步骤	PC	TV
1	获取 CI 卡功能状态	
2		返回 CI 卡功能状态

4.22 检查 IP 地址

发送获取 IP 地址命令(输出格式: XX:XX:XX:XX) (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	31	检查 IP 地址的指令
05	100-(data[2] + ... + data[05 - 2])	校验码

返回 IP 地址(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN-1	包长度
03	03	工厂自动测试协议
04	32	返回 IP 地址的指令
05	00-FF	IP 地址字串
...	00-FF	IP 地址字串
NN-1	00-FF	IP 地址字串
NN	100-(data[2] + ... + data[NN - 2])	校验码

流程:

步骤	PC	TV
1	获取 IP 指令	
2		返回 IP 命令

4.23 检查 WIFI

发送 WIFI 检查命令 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	33	检查 WIFI 指令
05	100-(data[2] + ... + data[05 - 2])	校验码

返回 WIFI 测试结果(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度

03	03	工厂自动测试协议
04	34	返回 WIFI 测试结果
05	00/01/02	0: 正常; 1: 正在检查; 2: 失败
06	100-(data[2] + ... + data[NN - 2])	校验码

流程:

步骤	PC	TV
1	检查 WIFI 指令	
2		WIFI 检查结果

4.23 检查 USB 功能

发送 USB 检查命令 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	35	检查 USB 接口指令
05	100-(data[2] + ... + data[05 - 2])	校验码

返回 USB 测试结果(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	36	返回 USB 测试结果
05	FF	USB 连接个数
06	100-(data[2] + ... + data[NN - 2])	校验码

流程:

步骤	PC	TV
----	----	----

1	检查 USB 指令	
2		USB 检查结果

4.24 检查 BLUETOOTH 功能

发送检查蓝牙指令 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	38	检查蓝牙指令
05	100-(data[2] + ... + data[05 - 2])	校验码

TV 返回蓝牙测试状态 (TV-PC)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	39	返回蓝牙测试结果
05	00/01/02	0: 正常; 1: 正在检查; 2: 失败
06	100-(data[2] + ... + data[NN - 2])	校验码

流程

步骤	PC	TV
1	检查蓝牙指令	
2		返回 BULETOOCH 测试结果

4.25 发送遥控指令

PC 发送遥控码值 (PC->TV)

字节序号	值	描述
------	---	----

00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	37	遥控操作指令
05	NN	工厂遥控器按键码值
06	100-(data[2] + ... + data[05 - 2])	校验码

TV 在接收改指令后，立刻返回 ACK，再执行遥控实际指令 (TV->PC)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0C	包长度
03	03	工厂自动测试协议
04	01	ACK CMD ID
05	00-02	ACK ERROR CODE
06	37	Cmd ID for ACK，返回应答的 CMD ID
07	00	Pocket Idx (31:24)
08	00	Pocket Idx (23:16)
09	00	Pocket Idx (15:8)
0A	00	Pocket Idx (7:0)
0B	100-(data[2] + ... + data[0B - 1])	校验码

流程

步骤	PC	TV
1	检查遥控器指令	
2		返回 ACK，告知 PC，TV 已经接收到指令

4.26 发送心跳指令

TV 在执行长时间处理时，需要返回心跳信号 (ACK 返回值 0xBBBBBBBB)，证明 TV 还在工作。心跳指令使用特殊的 ACK 返回 (TV->PC)

字节序号	值	描述
------	---	----

00	FF	同步字
01	33	开始字
02	0C	包长度
03	03	工厂自动测试协议
04	01	ACK CMD ID
05	00-02	ACK ERROR CODE
06	NN	Cmd ID for ACK, 返回当前正在处理的 CMD ID
07	BB	Pocket Idx (31:24)
08	BB	Pocket Idx (23:16)
09	BB	Pocket Idx (15:8)
0A	BB	Pocket Idx (7:0)
0B	100-(data[2] + ... + data[0B - 1])	校验码

4.27 文件传输指令

开始传输指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0F	包长度
03	03	工厂自动测试协议
04	40	开始提示传输文件 的 CMD
05	00-FF	FILE ID [31 : 24]
06	00-FF	FILE ID [23 : 16]
07	00-FF	FILE ID [15 : 8]
08	00-FF	FILE ID [7 : 0]
9	00-FF	FILE SIZE[31 : 24]
0A	00-FF	FILE SIZE[23 : 16]
0B	00-FF	FILE SIZE[15 : 8]
0C	00-FF	FILE SIZE[7 : 0]
0D	00-FF	文件类型 ID HDCP1.x = 1 CI Plus = 2 HDCP2.0 = 3

		HDCP2.2 = 4 Widevine Key = 5 ESN Key= 6 Device ID = 7 Roku Key = 8 (烧录本地固定Key) DevCert = 9 (Iot Key) Attestation = 10; ULPK = 11; Playready = 12; Google = 13; NetflixKey = 14; CMEIkey = 15; Ternary = 16; MGK = 17; HashKey = 18; for Oppo tuyaIotKey = 19; CI Plus 2nd = 20; Fairplay = 21; HDCPTX 1.4 = 22; HDCPTX 2.2 = 23; SMART HOME = 24; MIRACAST = 25; CommonType1 = 26;(烧录本地固定Key) CommonType2 = 27;(烧录本地固定Key) CommonType3 = 28;(烧录本地固定Key) CommonType4 = 29;(烧录本地固定Key) PEK = 30; YouTube = 31; EDV Key =32; KFP = 33; OTA Client = 34; LEK Key = 35; NabtoKey = 36; TivoKey = 37; PFID = 38; PFPK = 39; MarlinKey = 40; RMCA key = 98; CombinedKey = 99;
0E	100-(data[2] + ... + data[0E-1])	校验码

返回是否能传输指令 (TV->PC)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	09	包长度
03	03	工厂自动测试协议
04	41	返回 FILE 可以传输的应答 CMD
05	00-02	FILE_STATUS (0:NORMAL,1:ALREADY EXIST, 2: Useless)
06	00-FF	MAX PACKET LENGTH [15:8]
07	00-FF	MAX PACKET LENGTH [7:0]
08	100-(data[2] + ... + data[08-1])	校验码

传输数据指令(PC->TV):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	42	FILE 数据包 CMD
05	00-FF	POCKET INDEX [31 : 24]
06	00-FF	POCKET INDEX [23 : 16]
07	00-FF	POCKET INDEX [15 : 8]
08	00-FF	POCKET INDEX [7 : 0]
09	00-FF	Pocket Total count [31 : 24]
0A	00-FF	Pocket Total count [23 : 16]
0B	00-FF	Pocket Total count [15 : 8]
0C	00-FF	Pocket Total count [7 : 0]
0D	00-FF	DATA
...	...	...
NN-2	00-FF	DATA
NN-1	100-(data[2] + ... + data[NN-2])	校验码

对接收到的数据进行应答 ACK 指令 (TV->PC) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0C	包长度
03	03	工厂自动测试协议
04	01	ACK CMD ID
05	00-02	ACK ERROR CODE
06	42	Cmd ID for ACK, 返回应答的 CMD ID
07	00-FF	Pocket Idx (31:24)
08	00-FF	Pocket Idx (23:16)
09	00-FF	Pocket Idx (15:8)
0A	00-FF	Pocket Idx (7:0)
0B	100-(data[2] + ... + data[0B - 1])	校验码

ACK 返回的 Pocket idx 为接收到的包序号。

发送 file CRC 指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	08	包长度
03	03	工厂自动测试协议
04	43	发 file crc 的 CMD
05	00-FF	CRC [15 : 8]
06	00-FF	CRC [7 : 0]
07	100-(data[2] + ... + data[07 - 1])	校验码

返回烧录结果指令 (TV->PC) :

字节序号	值	描述
00	FF	同步字
01	33	开始字

02	07	包长度
03	03	工厂自动测试协议
04	44	返回 FILE 烧录状态的 CMD
05	00-02	FILE STATUS(0: OK, 1: CRC ERROR, 2:write flash error)
06	100-(data[2] + ... + data[07 - 1])	校验码

流程:

步骤	PC	TV	工作内容
1	START_SEND_FILE		传送 FILE 文件编码 ID 和文件类型, TV 收到后会判断是否可以烧录该文件, 如果可以, 申请空间, 然后由步骤 2 返回应答
2		RET_START_SEND_FILE	返回: 能否烧录以及一个 pocket 最多可以传多少 byte 的 file 数据
3	SEND_FILE_DATA		传输数据
4		ACK	返回是否正确以及当前时第几个 pocket
...	...	...	...
N	SEND_FILE_DATA		
N+1		ACK	
N+2	SEND_FILE_CRC		当 n+1 的 ACK 返回的当前包等于总包数时, 表示发完数据, PC 发 CRC 码到 TV, TV 校验, 如果一致, 则写 flash
N+3		ACK_FILE_STATUS	返回: 1, CRC 是否一致 2, 写入是否成功

4.28 获取指定类型 FILE ID

获取指定类型 FILE ID 的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字

02	07	包长度
03	03	工厂自动测试协议
04	45	获取 FILE ID CMD HDCP1.x = 1 CI Plus = 2 HDCP2.0 = 3 HDCP2.2 = 4 Widevine Key = 5 ESN Key= 6 Device ID = 7 Roku Key = 8 (烧录本地固定 Key) DevCert = 9 (Iot Key) Attestation = 10; ULPK = 11; Playready = 12; Google = 13; NetflixKey = 14; CMEIkey = 15; Ternary = 16; MGK = 17; HashKey = 18; for Oppo tuyaIotKey = 19; CI Plus 2nd = 20; Fairplay = 21; HDCPTX 1.4 = 22; HDCPTX 2.2 = 23; SMART HOME = 24; MIRACAST = 25; CommonType1 = 26;(烧录本地固 定Key) CommonType2 = 27;(烧录本地固 定Key) CommonType3 = 28;(烧录本地固 定Key) CommonType4 = 29;(烧录本地固 定Key) PEK = 30; YouTube = 31; EDV Key =32; KFP = 33; OTA Client = 34; LEK Key = 35; NabtoKey = 36; TivoKey = 37;

		PFID = 38; PFPK = 39; MarlinKey = 40;
05	00-FF	文件类型
06	100-(data[2] + ... + data[06 - 1])	校验码

返回文件 ID 指令(TV->PC):

字节序号	值	描述
00	FF 有没上拉，有上拉去掉，或者在小板上面做个下拉	同步字
01	33	开始字
02	0B	包长度
03	03	工厂自动测试协议
04	46	返回 file id
05	00-FF	FILE ID [31 : 24]
06	00-FF	FILE ID [23 : 16]
07	00-FF	FILE ID [15 : 8]
08	00-FF	FILE ID [7 : 0]
09	00-FF	文件类型
0A	100-(data[2] + ... + data[0A - 1])	校验码

流程:

步骤	PC	TV
1	发送获取指定 FILE 类型的 ID	
2		返回 FILE ID 和类型

4.29 设置 KEYPAD 逻辑功能开关

Set 后，KEYPAD 的逻辑功能关闭，以便读取 KEY 的被按下的状态，Reset 后，逻辑功能恢复。

设置指令 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度

03	03	工厂自动测试协议
04	47	CTRL KEY PAD CMD ID
05	00-01	00:unlock keypad 逻辑功能 01: lock keypad 逻辑功能
06	100-(data[2] + ... + data[06 - 1])	校验码

返回为 ACK 变种，由 TV 给出是否成功设置（TV->PC）

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0C	包长度
03	03	工厂自动测试协议
04	01	ACK CMD ID
05	00-02	ACK ERROR CODE
06	47	Cmd ID for ACK, 返回应答的 CMD ID
07	00	--
08	00	--
09	00	--
0A	00-01	00: 当前 keypad 逻辑功能 unlock 01: 当前 keypad 逻辑功能 lock
0B	100-(data[2] + ... + data[0B - 1])	校验码

流程:

步骤	PC	TV
1	发送 lock CMD	
2		返回 ACK
3	测试 key pad	
4	发送 unlock cmd	
5		返回 ACK

4.30 设置 Panel model index

该功能主要是面向一些板卡需要 Panel 序号初始化，在出厂前设置。设置 Panel model index（PC->TV）：

字节序号	值	描述
------	---	----

00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	48	设置 Panel model index 指令
05	00-FF	Panel model index(外部文件配置)
06	100-(data[2] + ... + data[06 - 1])	校验码

返回 ACK 应答(TV->PC)

流程：

步骤	PC	TV
1	SET_PANEL_MODEL_ID	
2		ACK

4.31 获取 Panel model index

该功能主要是面向一些板卡需要 Panel 序号初始化，在出厂前设置。设置 Panel model index (PC->TV)：

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	49	获取 Panel model index 指令
05	100-(data[2] + ... + data[05 - 1])	校验码

返回 ACK 应答(TV->PC),包含返回的 Panel model id:

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0C	包长度
03	03	工厂自动测试协议
04	01	ACK CMD ID

05	00-02	ACK ERROR CODE
06	49	Cmd ID for ACK, 返回应答的 CMD ID
07	00	N/A
08	00	N/A
09	00	N/A
0A	00-FF	获取到的 Panel model id
0B	100-(data[2] + ... + data[0B - 1])	校验码

流程:

步骤	PC	TV
1	GET_PANEL_MODEL_ID	
2		ACK

4.32 烧录 TCL Device ID

注意: Device ID 的长度为 40

设置 Device ID 指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	0x50	设置 Device ID 的指令
05	XX	data
...	...	...
NN-1	XX	data
NN	100-(data[2] + ... + data[NN - 1])	校验码

流程:

步骤	PC	TV
1	设置 Device ID	
2		返回 ACK

4.33 获取 Device ID 条形码

获取 Device ID 的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	0x51	获取板卡 Device ID 的 CMD
05	100-(data[2] + ... + data[06 - 1])	校验码

返回 Device ID 的指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	0x52	设置获取 Device ID 的指令
05	XX	data
...	...	...
NN-1	XX	data
NN	100-(data[2] + ... + data[NN - 1])	校验码

获取流程

步骤	PC	TV
1	获取 TCL Device ID	
2		返回 TCL Device ID

4.34 获取 USB 设备状态

获取 USB 设备状态的指令 (PC->TV) :

发送获取指令之后依据回传的命令长度判断目前板卡有几个 USB 设备可以使用。

并根据回传的数值: 0x00=当前的 USB 端口正常, 0x01=当前的 USB 端口异常

字节序号	值	描述
------	---	----

00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	0x53	获取板卡所有 USB 设备状态的 CMD
05	100-(data[2] + ... + data[06 - 1])	校验码

返回 USB 设备序列(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0x1A	包长度, USB 设备共 20 个
03	03	工厂自动测试协议
04	0x54	设置获取 USB 设备状态的指令
05	XX	USB-1 设备状态 0x00:USB2.0 插入 USB 盘且正常 0x01:USB2.0 未插入 USB 盘或异常 0x02:默认状态, 表示该设备状态未确定 0x03:SD 插入 SD 卡且正常 0x04:SD 未插入 SD 卡或异常 0x05:USB3.0 插入 USB 盘且正常 0x06:USB3.0 未插入 USB 盘或异常 0x07:WIFI-USB 设备正常 0x08:WIFI-USB 设备错误 0x09:BT-USB 设备正常 0x0A:BT-USB 设备错误
...	...	...
NN-1	XX	USB-1 设备状态 0x00:USB2.0 插入 USB 盘且正常 0x01:USB2.0 未插入 USB 盘或异常 0x02:默认状态, 表示该设备状态未确定

		0x03:SD 插入 SD 卡且正常 0x04:SD 未插入 SD 卡或异常 0x05:USB3.0 插入 USB 盘且正常 0x06:USB3.0 未插入 USB 盘或异常 0x07:WIFI-USB 设备正常 0x08:WIFI-USB 设备错误 0x09:BT-USB 设备正常 0x0A:BT-USB 设备错误
NN	100-(data[2] + ... + data[NN - 1])	校验码

获取流程

步骤	PC	TV
1	获取板卡上所有 USB 设备状态	
2		返回 USB 设备状态数组

4.35 设置通用指令并返回

通用指令是指工厂可以根据需求自行定制的指令，主要用来灵活处理抓取客户特殊数据或客户定制数据写入等功能，需要 TV 的软件配合工厂命令一起设计。

如果使用本命令，需要与工厂直接沟通，并在测试指南中注明使用的方法

设置通用指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度，最大到 0xFF
03	03	工厂自动测试协议
04	0x55	设置通用指令
05	NN	0x00~0xFF 设置通用指令序号
06	XX	传入的字符串以 ASCII 码表示
...	...	...
NN-1	XX	data
NN	100-(data[2] + ... + data[NN - 1])	校验码

返回通用指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度，最大到 0xFF
03	03	工厂自动测试协议
04	0x56	设置通用指令的回传
05	NN	0x00~0xFF 设置通用指令序号
06	NN	返回值的类型 0x00 Boolean 型判断（返回字符串固定为 1 或 0） 0x01 判断返回字符串的唯一性 0x02 判断返回字符串与 PC 端预设字符串相同 0x03 判断返回字符串与 PC 端预设字符串返回之内 0x04 判断返回字符串是否包含预设字符串
07	XX	回传的字符串以 ASCII 码表示
...	...	...
NN-1	XX	data
NN	100-(data[2] + ... + data[NN - 1])	校验码

获取流程

步骤	PC	TV
1	设置通用命令	
2		返回通用命令处理之后的数据

4.36 获取固定版本号 TV Checksum

注：后续均为 16 进制

获取命令(PC->TV):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	57	获取固定版本号 checksum id

05	E5	校验码
----	----	-----

返回命令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	58	返回固定版本号 checksum
05...(NN-2)	00-FF	CHECKSUM 数据
NN-1	100-(data[2] + ... + data[NN-2])	校验码

流程:

步骤	PC	TV
1	GET_CHECKSUM	
2		RET_CHECKSUM

4.37 烧录可配置 MAC 地址

MAC 地址为: $D_0 : D_1 : D_2 : D_3 : D_4 : D_5$,

烧录的格式为: $D_0D_1D_2D_3D_4D_5$

烧录指令 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0D	包长度
03	03	工厂自动测试协议
04	59	写入 MAC 的 CMD
05	00-FF	0x00 -- Ethernet MAC 0x01 -- WIFI MAC 0x02 -- Bluetooth MAC 0x03 -- Customer1 MAC 0x04 -- Customer2 MAC 0x05 -- Customer3 MAC
06	00-FF	D_0

07	00-FF	D ₁
08	00-FF	D ₂
09	00-FF	D ₃
0A	00-FF	D ₄
0B	00-FF	D ₅
0C	100-(data[2] + ... + data[0B - 1])	校验码

流程:

步骤	PC	TV
1	烧录指令 MAC	
2		返回 ACK

4.38 获取可配置 MAC 地址

获取 MAC 地址指令 (PC-TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	5A	获取 MAC 的指令
05	00-FF	0x00 -- Ethernet MAC 0x01 -- WIFI MAC 0x02 -- Bluetooth MAC 0x03 -- Customer1 MAC 0x04 -- Customer2 MAC 0x05 -- Customer3 MAC
06	100-(data[2] + ... + data[05 - 2])	校验码

返回 MAC 指令 (TV->PC) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0D	包长度
03	03	工厂自动测试协议
04	5B	返回 MAC 的 CMD

05	00-FF	0x00 -- Ethernet MAC 0x01 -- WIFI MAC 0x02 -- Bluetooth MAC 0x03 -- Customer1 MAC 0x04 -- Customer2 MAC 0x05 -- Customer3 MAC
06	00-FF	D ₀
07	00-FF	D ₁
08	00-FF	D ₂
09	00-FF	D ₃
0A	00-FF	D ₄
0B	00-FF	D ₅
0C	100-(data[2] + ... + data[0B - 1])	校验码

流程:

步骤	PC	TV
1	获取 MAC 指令	
2		返回 MAC 指令

4.39 烧录用户自定义 Barcode 条形码

注意: BARCODE 的长度不固定

设置 Customer Barcode 指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	5C	设置用户自定义 barcode 的指令
05	XX	用户定义的 Barcode 类型 CUS1 = 0 (Customer barcode) CUS2 = 1 (ESN Key FileName) CUS3 = 2 CUS4 = 3 CUS5 = 4 CUS6 = 5 (Smart 烧录核心板客户条码专用) 扩展功能:

		0x80 WideVine Key FileName 注意: 新方案 ESN 与 Widevine Key 烧录请使用文件传输指令, 此处保留仅为兼容旧方案
06	XX	barcode-data
...	...	...
NN-1	XX	barcode-data
NN	100-(data[2] + ... + data[NN - 1])	校验码

返回 ACK 应答(TV->PC)

流程:

步骤	PC	TV
1	设置 barcode	
2		返回 ACK

4.40 获取用户自定义 Barcode 条形码

获取 Customer barcode 的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	5D	获取用户自定义板卡 BARCODE 的 CMD
05	XX	用户定义的 Barcode 类型 CUS1 = 0 (Customer barcode) CUS2 = 1 (ESN Key FileName) CUS3 = 2 CUS4 = 3 CUS5 = 4 CUS6 = 5 (Smart 烧录核心板客 户条码专用) 扩展功能: 0x80 WideVine Key FileName 注意: 新方案 ESN 与 Widevine Key 烧录请使用文件传输指令, 此处保留仅为兼容旧方案

06	100-(data[2] + ... + data[06 - 1])	校验码
----	------------------------------------	-----

返回 BARCODE 的指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	5E	返回用户自定义 barcode 的指令
05	XX	用户定义的 Barcode 类型 CUS1 = 0 (Customer barcode) CUS2 = 1 (ESN Key FileName) CUS3 = 2 CUS4 = 3 CUS5 = 4 扩展功能: 0x80 WideVine Key FileName 注意: 新方案 ESN 与 Widevine Key 烧录请使用文件传输指令, 此处保留仅为兼容旧方案
06	XX	barcode-data
...	...	...
NN-1	XX	barcode-data
NN	100-(data[2] + ... + data[NN - 1])	校验码

获取流程

步骤	PC	TV
1	获取用户自定义 barcode	
2		返回用户自定义 barcode

4.41 获取老化模式状态

获取老化模式状态的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字

02	07	包长度
03	03	工厂自动测试协议
04	5F	获取老化模式状态
05	XX	返回值类型 AGING_TIME = 0 AGING_POWERON_TIMES = 1 TV_POWERON_TIME = 2 NOW_POWERON_TIME = 3 POWERON_TIMES = 4 0 = 板卡老化模式 On 之后到获取时的总时间 1 = 板卡老化模式 On 之后的开机次数, 在老化模式 Off 时清零 2 = 电视从第一次开机到获取参数经过的总时间 3 = 本次开机到获取值经过的时间 4 = 电视从首次开机到获取参数时的总开机次数
06	100-(data[2] + ... + data[06 - 1])	校验码

返回老化模式状态的指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0B	包长度
03	03	工厂自动测试协议
04	60	返回老化模式状态的指令
05	XX	返回值类型 AGING_TIME = 0 AGING_POWERON_TIMES = 1 TV_POWERON_TIME = 2 NOW_POWERON_TIME = 3 POWERON_TIMES = 4 0 = 板卡老化模式 On 之后到获取时的总时间 1 = 板卡老化模式 On 之后的开机次数, 在老化模式 Off 时清零

		2 = 电视从第一次开机到获取参数经过的总时间 3 = 本次开机到获取值经过的时间 4 = 电视从首次开机到获取参数时的总开机次数
06	0x00 ~ 0xFF	覆盖范围为 0x00000000 ~ 0xFFFFFFFF 对数据位关系为 [06 07 08 09] 所有时间单位都是（分钟）
07	0x00 ~ 0xFF	
08	0x00 ~ 0xFF	
09	0x00 ~ 0xFF	
0A	100-(data[2] + ... + data[NN - 1])	校验码

获取流程

步骤	PC	TV
1	获取板卡老化模式状态	
2		返回板卡老化模式状态

4.42 烧录用户自定义 ProjectID 条形码

注意：ProjectID 是变长的

设置 Customer ProjectID 指令（PC->TV）：

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	61	设置用户自定义 ProjectID 的指令
05	XX	用户定义的 ProjectID 类型 CUS1 = 0 CUS2 = 1 CUS3 = 2 CUS4 = 3 CUS5 = 4
06	XX	data
...	...	...
NN-1	XX	data

NN	100-(data[2] + ... + data[NN - 1])	校验码
----	------------------------------------	-----

返回 ACK 应答(TV->PC)

流程:

步骤	PC	TV
1	设置 ProjectID	
2		返回 ACK

4.43 获取用户自定义 ProjectID 条形码

获取 Customer ProjectID 的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	62	获取用户自定义板卡 ProjectID 的 CMD
05	XX	用户定义的 ProjectID 类型 CUS1 = 0 CUS2 = 1 CUS3 = 2 CUS4 = 3 CUS5 = 4
06	100-(data[2] + ... + data[06 - 1])	校验码

返回 BARCODE 的指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	63	返回用户自定义 ProjectID 的指令
05	XX	用户定义的 ProjectID 类型 CUS1 = 0

		CUS2 = 1 CUS3 = 2 CUS4 = 3 CUS5 = 4
06	XX	data
...	...	...
NN-1	XX	data
NN	100-(data[2] + ... + data[NN - 1])	校验码

获取流程

步骤	PC	TV
1	获取用户自定义 ProjectID	
2		返回用户自定义 ProjectID

4.44 传输烧录文件的文件名

传输数据指令(PC->TV):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	64	FILE 数据包 CMD
05	00-FF	文件类型 ID HDCP1.x = 1 CI Plus = 2 HDCP2.0 = 3 HDCP2.2 = 4 Widevine Key = 5 ESN Key = 6 Device ID = 7 Roku Key = 8 (烧录本地固定 Key) DevCert = 9 (Iot Key) Attestation = 10; ULPK = 11; Playready = 12;

06	XX	Google = 13; NetflixKey = 14; CMEIkey = 15; Ternary = 16; MGK = 17; HashKey = 18; for Oppo tuyaIotKey = 19; CI Plus 2nd = 20; Fairplay = 21; HDCPTX 1.4 = 22; HDCPTX 2.2 = 23; SMART HOME = 24; MIRACAST = 25; CommonType1 = 26;(烧录本地 固 定Key) CommonType2 = 27;(烧录本地 固 定Key) CommonType3 = 28;(烧录本地 固 定Key) CommonType4 = 29;(烧录本地 固 定Key) PEK = 30; YouTube = 31; NabtoKey = 36; Bin Name ASCII Code
...	...	...
NN-1	XX	Bin Name ASCII Code
NN	100-(data[2] + ... + data[NN - 1])	校验码

返回 ACK 应答(TV->PC)

流程:

步骤	PC	TV
1	传输烧录文件的文件名	
2		返回 ACK

4.28 获取烧录文件的文件名

获取指定类型 FILE NAME 的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字

02	07	包长度
03	03	工厂自动测试协议
04	65	获取 FILE NAME CMD
05	00-FF	文件类型 HDCP1.x = 1 CI Plus = 2 HDCP2.0 = 3 HDCP2.2 = 4 Widevine Key = 5 ESN Key = 6 Device ID = 7 Roku Key = 8 (烧录本地固定Key) DevCert = 9 (Iot Key) Attestation = 10; ULPK = 11; Playready = 12; Google = 13; NetflixKey = 14; CMEIkey = 15; Ternary = 16; MGK = 17; HashKey = 18; for Oppo tuyalotKey = 19; CI Plus 2nd = 20; Fairplay = 21; HDCPTX 1.4 = 22; HDCPTX 2.2 = 23; SMART HOME = 24; MIRACAST = 25; CommonType1 = 26;(烧录本地固定Key) CommonType2 = 27;(烧录本地固定Key) CommonType3 = 28;(烧录本地固定Key) CommonType4 = 29;(烧录本地固定Key) PEK = 30; YouTube = 31; NabtoKey = 36;
06	100-(data[2] + ... + data[06 - 1])	校验码

返回文件 ID 指令(TV->PC):

字节序号	值	描述
00	FF 有没上拉，有上拉去掉，或者在小板上面做个下拉	同步字
01	33	开始字
02	0B	包长度
03	03	工厂自动测试协议
04	66	返回 FILE NAME CMD
05	00-FF	文件类型 HDCP1.x = 1 CI Plus = 2 HDCP2.0 = 3 HDCP2.2 = 4 Widevine Key = 5 ESN Key= 6 Device ID = 7 Roku Key = 8 (烧录本地固定 Key) DevCert = 9 (Iot Key) Attestation = 10; ULPK = 11; Playready = 12; Google = 13; NetflixKey = 14; CMEIkey = 15; Ternary = 16; MGK = 17; HashKey = 18; for Oppo tuyalotKey = 19; CI Plus 2nd = 20; Fairplay = 21; HDCPTX 1.4 = 22; HDCPTX 2.2 = 23; SMART HOME = 24; MIRACAST = 25; CommonType1 = 26;(烧录本地固 定Key) CommonType2 = 27;(烧录本地固 定Key) CommonType3 = 28;(烧录本地固 定Key) CommonType4 = 29;(烧录本地固 定Key)

06	XX	PEK = 30; YouTube = 31; NabtoKey = 36; Bin Name ASCII Code
...	...	...
NN-1	XX	Bin Name ASCII Code
NN	100-(data[2] + ... + data[NN - 1]) 校验码	

流程：

步骤	PC	TV
1	发送获取指定 FILE 类型的 NAME	
2		返回 FILE NAME 和类型

4.29 音频控制协议

注意：音频控制协议的长度不固定

音频控制协议是在原有 Cvte 的串口通信协议上的扩展，考虑到方案众多，参数各异。

因此音频控制协议的传输方式全部为单向的纯字符串设计，主要规格如下：

- 控制“用户菜单”中的 treble（高音）

“TREBLE:10”，“TREBLE:100”或者“TREBLE:-50”等，考虑到用户菜单中参数范围不一致，因此命令以 treble 开头，以“:”分号作为分隔符，后面的数字按各平台差异自行设计。

- 控制“用户菜单”中的 bass（重低音）

考虑到用户菜单中参数范围不一致，因此命令以 BASS 开头，以“:”分号作为分隔符，后面的数字按各平台差异自行设计，行为与 treble 类似。

- 控制“用户菜单”中的 EQ

因为 EQ 存在的阶梯和值连个属性，因此对 EQ 参数定义为：

“EQ:0,100”，“EQ:5,50”，“EQ:7,-20”等样式

EQ 最少为 1 阶，参数上从 0 开始，“EQ:0,xx”

参数分隔符为“，”逗号，逗号后面的数为该阶的实际值。考虑到值可能有负数，因此通信协议上传输的是字符串，具体行为由各方案自行定义。

- 控制“用户菜单”中的 AVL

AVL 的控制有两个属性：开，关。指令定义成“AVL:ON”，“AVL:OFF”。

- 播放 U 盘中的视频文件

音频测试需要测试 U 盘中的音视频文件，在协议设计上只给出需要播放的文件名称，具体的行为由软件自行设计，包括文件的后缀都要加上：

“USBPLAY:XXXXXXXXX.mp4”

“USBPLAY:XXXXXXXXX.AVI”

- 连接蓝牙设备

连接到某个蓝牙设备，这里给出的是连接到蓝牙设备的 SSID:

“BT:XXXX”

- 软开关机

测试过程中需要的软开关机，考虑到软开机实际上无法用串口命令实现，但也还是给出完整的命令组：

“POWER:ON”

“POWER:OFF”

音频控制协议的指令（PC->TV）：

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	67	设置音频调节的指令
05	XX	ASCII Code
...	...	...
NN-1	XX	...
NN	100-(data[2] + ... + data[NN - 1])	校验码

流程：

步骤	PC	TV
1	设置音频控制命令	

2	返回 ACK
---	--------

4.30 设置背光

设置背光命令(PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	68	设置背光指令
05	00-64	背光值
06	100-(data[2] + ... + data[06-1])	校验码

返回 ACK 应答(TV->PC)

流程:

步骤	PC	TV
1	SET_BACKLIGHT	
2		ACK

4.31 获取背光

发送背光获取命令 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	69	获取背光的指令
05	100-(data[2] + ... + data[05 - 2])	校验码

返回背光获取结果(TV->PC):

字节序号	值	描述
00	FF	同步字

01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	6A	返回当前背光值
05	FF	背光值
06	100-(data[2] + ... + data[NN - 2])	校验码

流程:

步骤	PC	TV
1	获取背光指令	
2		返回背光的值

4.32 获取复位状态

发送复位检查命令 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	6B	获取复位状态
05	100-(data[2] + ... + data[05 - 2])	校验码

返回复位检查结果(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	6C	返回当前复位状态
05	XX	复位状态 1 或 0
06	100-(data[2] + ... + data[NN - 2])	校验码

流程:

步骤	PC	TV
1	获取复位状态指令	
2		返回复位状态的值

4.33 获取音频测试值

发送获取音频自动化测试值命令 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	6D	获取音频自动化测试值
05	XX	设置的 ASCII, 第 1 个
NN-1	XX	设置的 ASCII, 第 NN-1 个
NN	100-(data[2] + ... + data[05 - 2])	校验码

返回获取音频自动化测试值结果(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	6E	返回音频自动化测试值
05	XX	返回值的 ASCII, 第 1 个
NN-1	XX	返回值的 ASCII, 第 NN-1 个
NN	100-(data[2] + ... + data[NN - 2])	校验码

流程:

步骤	PC	TV
1	获取音频测试返回值	
2		返回音频测试值的 ASCII

4.34 获取远场语音模块测试状态

发送获取远场语音模块测试状态值命令 (PC->TV)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	6E	获取远场语音模块测试值
05	100-(data[2] + ... + data[05 - 2])	校验码

返回获取远场语音模块测试状态结果(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	6F	返回远场语音模块测试值
05	XX	0: 正常; 1: 正在检查; 2: 失败
06	100-(data[2] + ... + data[NN - 2])	校验码

流程:

步骤	PC	TV
1	获取远场语音模块测试值	
2		返回远场语音模块测试值

4.40 获取按键板按键状态历史

获取按键板按键状态历史的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字

02	06	包长度
03	03	工厂自动测试协议
04	97	获取按键板按键状态历史的 CMD
06	100-(data[2] + ... + data[06 - 1])	校验码

返回按键板按键状态历史的指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	98	返回按键板按键状态历史的 CMD
06	XX	Key[0]
...	...	...
NN-1	XX	Key[N]
NN	100-(data[2] + ... + data[NN - 1])	校验码

获取流程

步骤	PC	TV
1	获取按键板状态历史	
2		返回按键板状态历史

4.41 获取 CPU 温度

获取 CPU 温度 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	71	获取 CPU 温度
06	100-(data[2] + ... + data[06 - 1])	校验码

返回 CPU 的温度(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	08	包长度
03	03	工厂自动测试协议
04	72	返回 CPU 温度值的 ASCII 码
06	XX	温度的十位 (ASCCI 码)
07	XX	温度的个位 (ASCCI 码)
08	100-(data[2] + ... + data[NN - 1])	校验码

获取流程

步骤	PC	TV
1	获取 CPU 温度	
2		返回 CPU 温度

4.42 启动小米的蓝牙扫描

启动小米的蓝牙扫描 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	17	包长度
03	03	工厂自动测试协议
04	73	启动蓝牙扫描
05	XX	MAC[0]
...	...	...
21	XX	MAC[16]
22	100-(data[2] + ... + data[06 - 1])	校验码

启动流程

步骤	PC	TV
1	启动小米蓝牙扫描	

4.43 获取 MCU 版本号

获取 MCU 版本号 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	86	获取 MCU 版本号
05	XX	ASCII Code
...	...	...
NN-1	XX	
NN	$100-(data[2] + \dots + data[06 - 1])$	校验码

返回 MCU 版本号(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	87	返回 MCU 版本号
05	XX	MCU VERSION
...	...	...
NN-1	XX	
NN	$100-(data[2] + \dots + data[06 - 1])$	校验码

获取流程

步骤	PC	TV
1	获取 MCU 版本号	
2		返回 MCU 版本号

4.44 获取所有 USB 口 U 盘名称

获取所有 USB 口 U 盘名称的指令 (PC->TV) :

发送获取指令之后依据回传的字符串和预期进行判断，完全匹配判定为正常，如果缺少哪个 U 盘名称，就排查对应的端子。

思路来源：Smart 客户要求我们精细化测试 U 盘，便于精准定位 U 盘异常位置。

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	0x99	获取板卡所有 USB 口 U 盘名称的 CMD
05	100-(data[2] + ... + data[06 - 1])	校验码

返回所有 USB 口 U 盘名称(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	0x9A	返回所有 USB 口 U 盘名称的指令
05	XX	返回格式：[usb name 1]+[usb name 2]+…… 比如：JU101 是 usb1 口，JU102 是 usb2 口，JU103 是 usb3 口，…… 要求：工厂插入到对应 usb 口的 U 盘必须按照端子丝印命名； 因此，组合后字符串如下： JU101+JU102+JU103+…… 返回值的 ASCII，第 1 个
NN-1	XX	返回值的 ASCII，第 NN-1 个
NN	100-(data[2] + ... + data[NN - 1])	校验码

获取流程

步骤	PC	TV
1	获取板卡上所有 USB 口 U 盘名称	
2		返回 USB 口 U 盘名称字符串

4.45 CVTE AT 执行 shell 指令

CVTE AT 执行 shell 指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	9B	执行 shell 的指令
05	XX	shell command
...	...	...
NN-1	XX	shell command
NN	100-(data[2] + ... + data[NN - 1])	校验码

流程:

步骤	PC	TV
1	shell 命令	
2		返回 ACK

4.46 烧录核心板 Barcode 条形码

注意: 核心板 BARCODE 的长度最大 60 字节, 设置和获取存在于核心板+底板组合形式的主板
设置核心板 Barcode 指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	9C	设置核心板 barcode 的 CMD
05	XX	barcode-data
...	...	...
NN-1	XX	barcode-data

NN	100-(data[2] + ... + data[NN - 1])	校验码
----	------------------------------------	-----

返回 ACK 应答(TV->PC)

流程:

步骤	PC	TV
1	设置核心板 barcode	
2		返回 ACK

4.47 获取核心板 Barcode 条形码

获取核心板 barcode 的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	9D	获取核心板 barcode 的 CMD
05	100-(data[2] + ... + data[06 - 1])	校验码

返回 BARCODE 的指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	9E	返回核心板 barcode 的 CMD
05	XX	barcode-data
...	...	...
NN-1	XX	barcode-data
NN	100-(data[2] + ... + data[NN - 1])	校验码

获取流程

步骤	PC	TV
----	----	----

1	获取核心板 barcode	
2		返回核心板 barcode

4.48 获取网络速率

获取网络速率的指令(PC->TV):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	A0	获取网络速率的 CMD
05	100-(data[2] + ... + data[05 - 1])	校验码

返回网络速率的指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	08	包长度
03	03	工厂自动测试协议
04	A1	返回网络速率的 CMD
05	FF	网络速率: 高 8 位
06	FF	网络速率: 低 8 位
07	100-(data[2] + ... + data[07 - 1])	校验码

获取流程

步骤	PC	TV
1	获取网络速率	
2		返回网络速率

4.48 通用文件回传指令

发送文件路径指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN + 1	包长度
03	03	工厂自动测试协议
04	A2	设置读取的文件路径指令
05	XX	板卡文件路径
...	...	...
NN-1	XX	板卡文件路径
NN	100-(data[2] + ... + data[NN - 1])	校验码

返回文件数据包数量指令 (TV->PC)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0A	包长度
03	03	工厂自动测试协议
04	A3	返回包索引
05	00-FF	Pocket Total count [31 : 24]
06	00-FF	Pocket Total count [23 : 16]
07	00-FF	Pocket Total count [15 : 8]
08	00-FF	Pocket Total count [7 : 0]
09	100-(data[2] + ... + data[09 - 1])	校验码

读取索引的数据包指令(PC->TV):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0E	包长度
03	03	工厂自动测试协议
04	A4	读取数据包 CMD
05	00-FF	Pocket Total count [31 : 24]
06	00-FF	Pocket Total count [23 : 16]

07	00-FF	Pocket Total count [15 : 8]
08	00-FF	Pocket Total count [7 : 0]
09	00-FF	POCKET INDEX [31 : 24]
0A	00-FF	POCKET INDEX [23 : 16]
0B	00-FF	POCKET INDEX [15 : 8]
0C	00-FF	POCKET INDEX [7 : 0]
0D	100-(data[2] + ... + data[0D - 1])	校验码

返回索引数据包指令 (TV->PC) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN + 1	包长度
03	03	工厂自动测试协议
04	A5	返回索引数据包指令
05	00-FF	Pocket Idx (31:24)
06	00-FF	Pocket Idx (23:16)
07	00-FF	Pocket Idx (15:8)
08	00-FF	Pocket Idx (7:0)
09	00-FF	DATA
...	...	DATA
NN - 1		DATA
NN	100-(data[2] + ... + data[NN - 1])	校验码

获取原始文件 CRC 指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度
03	03	工厂自动测试协议
04	A6	获取文件原始 CRC, 和上位机实际读取到的文件 CRC 校验
05	100-(data[2] + ... + data[05 - 1])	校验码

返回原始文件 CRC 指令 (TV->PC) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	08	包长度
03	03	工厂自动测试协议
04	A7	返回原始文件 CRC 指令
05	00-FF	CRC [15 : 8]
06	00-FF	CRC [7 : 0]
07	100-(data[2] + ... + data[07 - 1])	校验码

流程:

步骤	PC	TV	工作内容
1	SEND_FILE_PATH		发送要读取的文件路径
2		RET_FILE_PACKAGE_COUNT	返回文件数据包数量
3	READ_FILE_DATA		读取文件数据包
4		RET_FILE_DATA	返回当前索引的数据包
...	...	...	...
N	READ_FILE_DATA		读取文件数据包
N+1		RET_FILE_DATA	返回当前索引的数据包
N+2	READ_FILE_CRC		读取原始文件 CRC
N+3		RET_FILE_CRC	返回原始文件 CRC, PC 做校验

4.49 获取 RID

获取 rid 的指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	07	包长度
03	03	工厂自动测试协议
04	A8	获取 RID 的指令
05	00-FF	RID 类型

		SOC : 0x01 GSV2702: 0x02 RTD2172: 0x03
06	100-(data[2] + ... + data[06 - 1])	校验码

返回 RID 指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0C	包长度
03	03	工厂自动测试协议
04	A9	返回 RID 的指令
05	00-FF	RID 类型 SOC : 0x01 GSV2702: 0x02 RTD2172: 0x03
06	00-FF	D0
07	00-FF	D1
08	00-FF	D2
09	00-FF	D3
0A	00-FF	D4
0B	100-(data[2] + ... + data[0B - 1])	校验码

获取流程

步骤	PC	TV
1	获取 RID	
2		返回 RID

4.50 获取通道 RGB 值

获取通道 RGB 值指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	06	包长度

03	03	工厂自动测试协议
04	AA	获取通道 RGB 值的指令
05	100-(data[2] + ... + data[06 - 1])	校验码

返回通道 RGB 值指令(TV->PC):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	24	包长度
03	03	工厂自动测试协议
04	AB	返回通道 RGB 值的指令
05	00-FF	第一个点 R
06	00-FF	第一个点 G
07	00-FF	第一个点 B
08	00-FF	第二个点 R
09	00-FF	第二个点 G
0A	00-FF	第二个点 B
0B	00-FF	第三个点 R
0C	00-FF	第三个点 G
0D	00-FF	第三个点 B
0E	00-FF	第四个点 R
0F	00-FF	第四个点 G
10	00-FF	第四个点 B
11	00-FF	第五个点 R
12	00-FF	第五个点 G
13	00-FF	第五个点 B
14	00-FF	第六个点 R
15	00-FF	第六个点 G
16	00-FF	第六个点 B
17	00-FF	第七个点 R
18	00-FF	第七个点 G
19	00-FF	第七个点 B
1A	00-FF	第八个点 R

1B	00-FF	第八个点 G
1C	00-FF	第八个点 B
1D	00-FF	第九个点 R
1E	00-FF	第九个点 G
1F	00-FF	第九个点 B
20	00-FF	第十个点 R
21	00-FF	第十个点 G
22	00-FF	第十个点 B
23	100-(data[2] + ... + data[0B - 1])	校验码

获取流程

步骤	PC	TV
1	获取 RGB 值	
2		返回 RGB 值

4.51 烧录 Key 压缩包

开始传输 Key 压缩包指令 (PC->TV) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	AC	开始提示传输 Key 压缩包 的 CMD, 设置压缩包 Key 类型
05	00-FF	FILE SIZE[31 : 24]
06	00-FF	FILE SIZE[23 : 16]
07	00-FF	FILE SIZE[15 : 8]
08	00-FF	FILE SIZE[7 : 0]
09	00-FF	KEY_1 类型 ID, 参考 4.27 文件传输类型 ID
...	...	...
NN-2	00-FF	KEY_N 类型 ID

NN-1	100-(data[2] + ... + data[NN - 1])	校验码
------	------------------------------------	-----

返回是否支持烧录 Key 压缩包指令 (TV->PC)

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	09	包长度
03	03	工厂自动测试协议
04	AD	返回 KEY 压缩包可以传输的应答 CMD
05	00-02	FILE_STATUS (0:NORMAL,1:ALREADY EXIST, 2: Useless)
06	00-FF	MAX PACKET LENGTH [15:8]
07	00-FF	MAX PACKET LENGTH [7:0]
08	100-(data[2] + ... + data[08 - 1])	校验码

传输 Key 压缩包指令(PC->TV):

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	NN	包长度
03	03	工厂自动测试协议
04	AE	读取数据包 CMD
05	00-FF	POCKET INDEX [31 : 24]
06	00-FF	POCKET INDEX [23 : 16]
07	00-FF	POCKET INDEX [15 : 8]
08	00-FF	POCKET INDEX [7 : 0]
09	00-FF	Pocket Total count [31 : 24]
0A	00-FF	Pocket Total count [23 : 16]
0B	00-FF	Pocket Total count [15 : 8]
0C	00-FF	Pocket Total count [7 : 0]
0D	00-FF	DATA
...	...	...

NN-2	00-FF	DATA
NN-1	100-(data[2] + ... + data[NN - 2])	校验码

对接收到的数据进行应答 ACK 指令 (TV->PC) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	0C	包长度
03	03	工厂自动测试协议
04	01	返回索引数据包指令
05	00-02	ACK ERROR CODE
06	AE	Cmd ID for ACK, 返回应答的 CMD ID
07	00-FF	Pocket Idx (31:24)
08	00-FF	Pocket Idx (23:16)
09	00-FF	Pocket Idx (15:8)
0A	00-FF	Pocket Idx (7:0)
0B	100-(data[2] + ... + data[NN - 1])	校验码

ACK 返回的 Pocket idx 为接收到的包序号。

返回烧录结果指令 (TV->PC) :

字节序号	值	描述
00	FF	同步字
01	33	开始字
02	08	包长度
03	03	工厂自动测试协议
04	AF	返回组合 KEY 烧录结果指令的 CMD
05	00-02	FILE_STATUS (0:OK,1:CRC ERROR, 2: BURN NG)
06	00-FF	05 字节状态为 2 时, 烧录失败 Key 类型 ID

07	$100 - (\text{data}[2] + \dots + \text{data}[07 - 1])$	校验码
----	--	-----

流程:

步骤	PC	TV	工作内容
1	START_SEND_COMBINE_KEY		发起烧录请求，确定要烧录的 Key 类型
2		RET_START_SEND_COMBINE_KEY	返回是否支持对应的 Key 类型，以及一个 pocket 最多可以传多少 byte 的 file 数据
3	SEND_COMBINE_KEY_DATA		传输数据
4		RET_COMBINE_KEY_DATA	返回是否正确以及当前第几个 pocket
...	...	...	...
N	SEND_COMBINE_KEY_DATA		
N+1		RET_COMBINE_KEY_DATA	
N+2		ACK_COMBINE_KEY_STATUS	文件传输完成后，对 Key 进行解压缩后，一次性烧录多个 Key。

五、通用串口库介绍

接口库只实现接口，提供函数钩子名称，功能部分需要 TV 软件实现。

5.1 接口函数说明

以下函数需要 TV 软件实现。

函数名	参数/返回描述	功能描述
u8 (*CVTE_GetTVChecksum)(u8* checksum)	参数: checksum: 返回的 checksum, 返回: checksum 的长度	返回 TV 软件真实 checksum 字符串
u8 (*CVTE_GetTVFixedChecksum)(u8* checksum)	参数: checksum: 返回的 checksum, 返回: checksum 的长度	返回 TV 软件固定版本号 checksum 字符串
u8 (*CVTE_SetTVSource)(en_CVTE_AT_SOURCE source)	参数: source: 设置的通道 ID 返回: 0- OK, 1-FAIL	设置 TV 通道 en_CVTE_AT_SOURCE 为库提供的通道 ID
u8 (*CVTE_GetTVSource)(en_CVTE_AT_SOURCE *source)	参数: source: 返回的通道 ID 返回: 0- OK, 1-FAIL	获取 TV 板卡当前通道的 ID
u8 (*CVTE_SetVolume)(u8 volume)	参数: volume:音量大小, 0-100 返回: 0- OK, 1-FAIL	设置 TV 板卡音量
u8 (*CVTE_ChangeTVNum)(u16 num)	参数: num: 设置的台号 返回: 0- OK, 1-FAIL	切换频道 当在 ATV 就执行 ATV 的切换 当在 DTV 通道就只想 DTV 的切换 切换的台号非法时, 不执行动作
u8 (*CVTE_UsartSendData)(u8* datas, u16 len)	参数: datas : 输出的字符串 len: 字符串的长度 返回: 0- OK, 1-FAIL	调用 TV 板卡的串口输出函数, 输出本库的通信信息
u8 (*CVTE_GetTVMacAddr)(u8* macAddr)	参数: macAddr: 获取到的 MAC 地址 字符串 返回: 0- OK, 1-FAIL	获取 MAC 地址
u8 (*CVTE_GetTVHDCP_ID)(u32* hdecId)	参数: hdecId: 获取到的 hdec key id 返回: 0- OK, 1-FAIL	获取 HDCP KEY 的 ID, 该 ID 为 KEY 文件的编码
u8 (*CVTE_GetTVCIPlus_ID)(u32* ciplusId)	参数: ciplusId: 获取到的 ci+ key id 返回: 0- OK, 1-FAIL	获取 ci+ KEY 的 ID, 该 ID 为 KEY 文件的编码
u8 (*CVTE_SaveHDCPKeyToTV)(u8* key, u32 len, u32 fileId)	参数: key: 烧录内容 len: key 长度 fileId: 该 Key 的文件编码 返回: 0: 写入成功, 2: 写入失败	把获取到的 KEY 信息写入 flash 中
u8 (*CVTE_SaveCIPlusKeyToTV)(u8* key, u32 len, u32 fileId)	参数: key: 烧录内容 len: key 长度 fileId: 该 Key 的文件编码 返回: 0: 写入成功, 2: 写入失败	把获取到的 KEY 信息写入 flash 中
u8 (*CVTE_SaveMACAddrToTV)(u8* macAddr, u8 len)	参数: macAddr: 待写入的 MAC 地址 字符串, len: 该字符串长度 返回: 0: 写入成功, 2: 写入失败	写 MAC 地址到 flash
u8 (*CVTE_CtrlIOLevel)(en_CVTE_AT_IOPin pin, en_CVTE_AT_IO_LEVEL level)	参数: pin: 控制的 IO 口 level: IO 口的电平状态 返回: 0- OK, 1-FAIL	设置 TV 板卡的 IO 电平
u8 (*CVTE_GetCurrentKeyPadStatus)(u8* keyStatus)	参数: keyStatus: 按键状态 返回: 0- OK, 1-FAIL	通过一个字节[D7-D0]的内容代表按键板[K7-K0]的状态, D7 对应 K7, 其他如此一一对应
u8 (*CVTE_SaveBarcodeToFlash)(u8* barcode, u8 len);	参数: barcode 条形码数组, 末尾为'\0' len: 条形码长度 含'\0' 返回: 0- OK, 1-FAIL	写 barcode 动作
u8 (*CVTE_GetBarcodeFromFlash)(u8* retBarcode, u8* len);	参数: reBarcode 读出的条形码数组 len: 条形码长度, 不含'\0'	输出 barcode

	返回: 0- OK, 1-FAIL	
u8 (*CVTE_SaveCustomerBarcodeToFlash)(u8 type, u8* barcode, u8 len);	参数: type 用户自定义类型 barcode 条形码数组, 末尾为'\0' len: 条形码长度 含'\0' 返回: 0- OK, 1-FAIL	写用户自定义的 barcode 动作
u8 (*CVTE_GetBarcodeFromFlash)(u8type, u8* retBarcode, u8* len);	参数: type 用户自定义的类型 Barcode 读出的条形码数组 len: 条形码长度, 不含'\0' 返回: 0- OK, 1-FAIL	输出用户自定义的 barcode
u8 (*CVTE_GetHDCP_KSV_Code)(u8* ksvCode)	参数: ksvCode- 返回的 KSV 值 返回: 0- OK, 1-FAIL	读取 TV 板卡的 KSV 值
u8 (*CVTE_SetATSC_AIR_CABLE_mode)(u8 type);	参数: type 0-AIR, 1-CABLE 返回: 0- AIR, 1- CABLE	设置 ATSC 的模式
u8 (*CVTE_SetATSC_ProNumber)(u8 major, u8 minor);	参数: major-主频道号, minor-次频道号 返回: 0- OK, 1-FAIL	设置 ATSC 频点
u8 (*CVTE_GetCIFunctionStatus)(u8* status)	参数: status: 待返回的 CI 状态 返回: 0- OK, 1-FAIL	Status 值: 0 - OK; 1 - 未插入; 2 - CI 功能异常
u8 (*CVTE_GetIPAddr)(u8* ipStr);	参数: u8* ipStr: IP buffer 指针 返回: IP 串的长度	获取 IP 地址, 可以是 IPV6, 也可以是 IPV4
u8 (*CVTE_GetWifiTestResult)(u8* status);	参数: u8* status: 测试结果 返回: 0- OK, 1-FAIL	Status: 0 - ok; 1 - 正在测试; 2 - 测试失败
u8 (*CVTE_GetUSBConnectCount)(u8* usb_count);	参数: usb_count: 待返回 USB 连接个数 返回: 0- OK, 1-FAIL	返回 USB 连接个数
u8 (*CVTE_GetUSBConnectStatus)(u8* usb_count, u8* usb_status);	参数: usb_count: 待返回可工作 USB 设备个数 usb_status: 返回各 USB 设备状态数组 返回: 0- OK, 1-FAIL	返回板上所有 USB 设备的状态
u8 (*CVTE_CheckBluetoochStatus)(u8* status);	参数: u8* status: 测试结果 返回: 0- OK, 1-FAIL	Status: 0 - ok; 1 - 正在测试; 2 - 测试失败
u8 (*CVTE_FAC_IR_Ctrl)(u8 key_code);	参数: u8 key_code: 遥控按键码值 返回: 0- OK, 1-FAIL	遥控码值控制接口。建议直接调用 TV 方案的 CVTE 工厂遥控器处理函数
u8 (*CVTE_FAC_write_file)(u8* buf, u32 len, u32 fileId, u8 filetype);	参数: u8* buf: 数据指针 u32 len: 数据长度 u32 fileId: 文件唯一编码 u8 filetype: 文件类型 返回: 0- OK, 1-FAIL	写文件到 TV flash, 最大文件长度由 TV 方案决定。
U8 (*CVTE_FAC_get_file_id)(u8 filetype, u32* fileId);	参数: u8 filetype: 文件类型 u32* 返回的 ID 返回: 0-OK, 1-获取失败, 2-无该 FILETYPE ID	获取通用文件 ID
u8 (*CVTE_FAC_check_filetype)(u8 filetype, u8* result);	参数: u8 filetype: 文件类型	检查该方案的板卡程序是否需要烧录 filetype 类型的

	u8* 返回结果 返回: 0-OK, 1-获取失败	文件 result: 0-未烧录, 而且需要烧录, 1-已烧录, 2-不需烧录
u8(*CVTE_FAC_ctrl_keypad)(u8 enable, u8 *result);	参数: u8 enable: 0-unlock; 1-lock u8 *result: 返回设置后的状态, 0 代表是 unlock, 1 代表 lock 返回: 0-OK, 1-获取失败	lock 代表逻辑功能被关闭, 系统能识别到哪个按键被按下
u8(*CVTE_FAC_set_panel_model_id)(u8 index);	参数: u8 index 返回: 0-OK, 1-获取失败	必须写 flash 后才认为写入正常, 返回 OK
u8(*CVTE_FAC_get_panel_model_id)(u8 *index);	参数: u8 *index: 从 flash 读到的当前设置的 panel id 返回: 0-OK, 1-获取失败	
u8(*CVTE_SaveTCLDeviceIDToFlash)(u8* Deviceid, u8 len);	参数: Deviceid 数组, 末尾为'\0' len: 条形码长度 含'\0' 返回: 0- OK, 1-FAIL	写 Device ID 动作
u8(*CVTE_GetTCLDeviceIDFromFlash)(u8* retDeviceID, u8* len);	参数: retDeviceID 读出的条形码数组 len: 条形码长度, 不含'\0' 返回: 0- OK, 1-FAIL	输出 Device ID
u8(*CVTE_SendCommonDataToSystem)(u8 cmdtype, u8* inputdata, u8 inputlen, u8* outputdata, u8* outputlen, u8* outputtype);	参数: cmdtype 命令的类型 inputdata 数据的数据包 inputlen: 数据数据长度不含'\0' outputdata 输出数据包 outputlen 数据数据长度不含'\0' outputtype 返回值的类型 返回: 0- OK, 1-FAIL	通用命令的输入和输出数据规格
u8(*CVTE_GetConfigTVMacAddr)(u8 type, u8* macAddr)	参数: type: 数据类型 macAddr: 获取到的 MAC 地址 字符串 返回: 0- OK, 1-FAIL	获取可配置的 MAC 地址数据
u8(*CVTE_SaveConfigMACAddrToTV)(u8 type, u8* macAddr, u8 len)	参数: type: 数据类型 macAddr: 待写入的 MAC 地址 字符串, len: 该字符串长度 返回: 0: 写入成功, 2: 写入失败	写可配置的 MAC 地址到 flash
u8(*CVTE_GetAgingStatus)(u8 type, u32* status);	参数: type: 获取到的 status 返回: 0- OK, 1-FAIL	获取老化模式状态
u8(*CVTE_SaveCustomerProjectIDToFlash)(u8 type, u8* data, u8 len);	参数: type 用户自定义类型 data 条形码数组, 末尾为'\0' len: 条形码长度 含'\0' 返回: 0- OK, 1-FAIL	写用户自定义的 ProjectID 动作
u8(*CVTE_GetCustomerProjectIDFromFlash)(u8type, u8* data, u8* len);	参数: type 用户自定义的类型 data 读出的条形码数组 len: 条形码长度, 不含'\0' 返回: 0- OK, 1-FAIL	输出用户自定义的 ProjectID
u8(*CVTE_SaveFileNameToFlash)(u8 type, u8* name, u8 len);	参数: type 烧录文件类型 name 文件名称包括后缀, 末尾为'\0'	写烧录文件的文件名

	len: 条形码长度 含'\0' 返回: 0- OK, 1-FAIL	
u8 (*CVTE_GetFileNameToFlash)(u8 type, u8* retname, u8* len);	参数: type 烧录文件类型 retname 读出的条形码数组 len: 条形码长度, 不含'\0' 返回: 0- OK, 1-FAIL	读取烧录文件的文件名
u8 (*CVTE_ControlAudioAction)(u8* inputdata, u8 inputlen);	参数: inputdata 数据的数据包 inputlen: 数据数据长度不含'\0' 返回: 0- OK, 1-FAIL	音频控制指令
u8 (*CVTE_SetBacklight)(u8 val)	参数: val:背光大小, 0-100 返回: 0- OK, 1-FAIL	设置背光值
u8 (*CVTE_GetBacklight)(u8* val);	参数: val: 待返回当前设定的背光值 返回: 0- OK, 1-FAIL	返回背光值
u8 (*CVTE_GetResetStatus)(u8* val);	参数: val: 待返回当前设定的复位状态 返回: 0- OK, 1-FAIL	返回复位状态
u8 (*CVTE_GeAudioPro)(u8* val)	参数: val: 返回的音频测试的 ASCII, 返回: val 的长度	返回音频测试功能的 ASCII 值

附加的处理函数

函数名	描述	备注
void CVTE_HeartbeatPro(void);		心跳程序。当执行处理时间较长的函数时，TV 需要每隔一单位时间（如 1S）来调用该函数，以便告知上位机程序正在运行。
void CVTE_AT_setCmdProcessEnable(u8 enable);	参数: enable = true ,可以处理 = false, 不能处理	指令处理开关，第一次上电的时候均为开
u8 CVTE_AT_getCmdProcessEnable(void);	返回: true - 可以处理执行接收到的指令 false - 不能处理接收到的指令	返回指令处理开关状态

5.3 通用指令使用列表

目前有些方案占用了通用指令的数据位，因此后续方案添加的时候请告知任立嘉统一备案，如果需要新添加指令，请在这些命令之后添加，并且不能重复。邮箱: renlijia@cvte.com

方案	通用指令	作用
MSD628	01 getuuid	获取 Ali OS 的 UUID 返回长度为 42 的字符串
RTD2984T-CI+	02 getCHDeviceID	获取烧录在 RTD2984T-CI+方案中的长虹 DeviceID 数据，返回长度为 40 的字符串
V59	03 XX:XX:XX:XX:XX:XX	测试脚本，AT 软件读取 MAC 地址后，直接按 16 进制发出地址的内容， 0x11,0x22,0x33,0x44,0x55,0x66,0x00,0x00；后两位用 0x00 补齐，发送内容长度为 8 COMTEST 测试软件，点【蓝牙】获取适配器的地址，发送内容为 FF112233445566，需要在 MAC 地址前补上 FF，不分大小写 TV 软件解析并成功存下 MAC 地址，返回 1，

		否则返回 0
V59	04	获取当前蓝牙状态是否在播放状态，正在播放中，返回 1，其他所有状态，返回 0

之后所有新增通固定通用串口指令都从 0x80 开始

	80 getwidevine	获取 widevine bin 的唯一性校验数据
--	----------------	--------------------------

5.2 使用说明

该串口通信库提供一个全局的钩子函数结构体变量，涵盖 5.1 列出的所有钩子函数，库内部调用该全局变量来操作统一的接口，实现与方案无关。

全局钩子函数结构体变量：

```
extern st_CVTE_AT_RegFunction g_CVTEFuns;
```

调用步骤：

- 1，注册钩子函数。实现各个钩子函数的具体功能。
- 2，修改串口接收处理函数，在接收到命令字符串后，调用库函数的 void CVTE_AT_UsartProcessCmd(const void* datas)处理数据

例子：

实现获取 checksum 的函数：

```
u8 Mapp_GetTVChecksum(u8 *checksum);
```

然后注册钩子：

```
g_CVTEFuns.CVTE_GetTVChecksum = Mapp_GetTVChecksum;
```

当 TV 板卡接收到上位机传过来的串口指令，交给 CVTE_AT_UsartProcessCmd 处理，如果该指令是要获取 Checksum，那么程序会通过 CVTE_GetTVChecksum 接口钩子调用有具体功能的 Mapp_GetTVChecksum 函数来获取 Checksum。

注意：未调用或无需调用的接口函数，也务必注册，实现函数内不做任何处理。

例子：

如果无需实现：u8 (*CVTE_SaveMACAddrToTV)(u8* macAddr, u8 len)

则编写如下函数：

```
u8 Mapp_SaveMACAddrToTV(u8* macAddr, u8 len)
{
    return 1; //fail
}
```